

A Phase-Field Solver for Boiling Heat Transfer

Governing Equations, Numerical Methods, and a Python Prototype Toward 3D DNS

Adebanji Oluwatimileyin Adelowo

2026

Contents

1	Introduction and Motivation	4
1.1	Why boiling matters	4
1.2	Why direct numerical simulation	4
1.3	Why phase-field	5
1.4	This project in context	5
1.5	What this document is	6
2	The Problem: Multiphase Flow with Phase Change	7
2.1	The sharp-interface statement of the problem	7
2.2	What makes this numerically hard	7
3	Terminology and Foundations	9
3.1	Sharp interface versus diffuse interface	9
3.2	The order parameter	9
3.3	Interface thickness and its resolution	10
3.4	A survey of interface-capturing and interface-tracking methods	11
4	Mathematical Foundations of Phase-Field Methods	13
4.1	The Ginzburg–Landau free energy functional	13
4.2	The chemical potential and the variational derivative	14
4.3	Cahn–Hilliard versus Allen–Cahn	14
4.4	The sharp-interface limit	15
4.5	The conservative Allen–Cahn equation	15
5	Classical Approaches to Boiling Simulation	18
5.1	Front tracking	18
5.2	Volume of fluid	18
5.3	Level set	19
5.4	Phase field for phase change	19
6	Research Evolution	20
7	The Governing Equation Set	21
7.1	Conservative Allen–Cahn with vaporisation source — (R.1), (R.2)	21

7.2	Mass conservation with a phase-change source — (R.3)	21
7.3	One-fluid Navier–Stokes — (R.4), with mixture properties (R.5), (R.6)	22
7.4	Surface tension: the continuum surface force — (R.7), (R.8)	23
7.5	Energy equation with latent heat — (R.9), (R.10), (R.11)	23
7.6	Vaporisation rate from the Rankine–Hugoniot balance — (R.12)	24
7.7	Summary of the coupled system	25
8	Numerical Methods	26
8.1	Spatial discretisation	26
8.2	Why the pressure equation has constant coefficients	27
8.3	FFT solution of the Poisson equation	28
8.4	Time integration: the projection–correction scheme	29
8.5	Stability of the explicit scheme	30
9	Code Implementation	32
9.1	src/params.py — the parameter container	32
9.1.1	Defect 1: <code>gamma = eps</code> is dimensionally inconsistent	32
9.2	src/operators.py — spatial discretisation	33
9.3	src/phase_field.py — conservative Allen–Cahn	34
9.4	src/pressure.py — the FFT Poisson solver	34
9.5	src/flow.py — Navier–Stokes projection–correction	35
9.5.1	Defect 2: momentum advection uses the non-conservative form	36
9.5.2	Defect 3: the 3D viscous term drops the variable-viscosity correction	36
9.6	src/energy.py — energy equation and vaporisation rate	36
9.6.1	Defect 4: the latent-heat source does not match Eq. (R.11)	37
9.6.2	Defect 5: <code>mdot_from_heatflux_2d</code> is wrong, and vanishes identically when $k_v = k_l$	37
9.7	src/solver.py — the coupled time loop	38
9.7.1	Defect 6: the velocity clamp is N_x times looser than documented	39
9.7.2	Defect 7: unreachable code in <code>run_3d</code>	40
9.8	src/diagnostics.py — post-processing	40
10	Validation Experiments	42
10.1	Benchmark 1: 2D vapour bubble growth at constant vaporisation rate	42
10.2	Benchmark 2: 1D Stefan problem	43
11	Results and Interpretation	45
11.1	Bubble growth	45
11.2	The Stefan problem	47
12	Limitations	50
12.1	Vaporisation rate: no probe interpolation, and a broken library function	50
12.2	Explicit surface tension and the time-step restriction	50
12.3	Periodic boundary conditions only	50
12.4	No oscillation control	51
12.5	First-order explicit time integration	51
12.6	Structural defects carried forward	51
12.7	Collocated rather than staggered grid	52

12.8 Python performance	52
13 Future Directions and PhD Roadmap	53
13.1 Immediate corrections	53
13.2 Three-dimensional validation	53
13.3 Wall and outlet boundary conditions	53
13.4 Higher-order time integration	54
13.5 The Fortran/CUDA/MPI port	54
13.6 Physics and parameter studies	54
13.7 Roadmap summary	55
14 References	56
14.1 Software	58
14.2 Data availability	58

1 Introduction and Motivation

1.1 Why boiling matters

Boiling is the most effective passive mechanism available for removing heat from a surface. Because the latent heat of vaporisation of most working fluids exceeds their sensible heat capacity by two to three orders of magnitude — for water at atmospheric pressure, $h_{lv} = 2.25 \times 10^6$ J/kg against $C_{p,l} = 4216$ J/(kg · K), so a single kilogram of vaporised water absorbs as much energy as heating that kilogram by 534 K — a boiling surface can sustain heat fluxes that single-phase convection cannot approach. This is why boiling underpins nuclear reactor cores, steam power cycles, refrigeration loops, and the increasingly aggressive two-phase cooling systems used for high-power electronics and data-centre hardware.

The engineering difficulty is that the *design* of these systems still rests almost entirely on empirical correlations. Roccon (2025) opens on exactly this point: “the prediction of boiling heat transfer coefficients in these systems still largely relies on empirical correlations, often leading to unsatisfactory results.” The reason is structural, not a matter of insufficient curve-fitting. Boiling is intrinsically multiscale. The chain of physics runs from the molecular scales that govern nucleation of a vapour embryo, through the micrometre-scale liquid microlayer trapped beneath a growing bubble, to the millimetre-scale bubble that detaches and rises, and finally to the centimetre- and metre-scale turbulent flow that carries it away, deforms it, and drives its coalescence or breakup. A correlation fitted at one point in that parameter space has no principled reason to extrapolate.

Two quantities in particular resist correlation. The first is the **critical heat flux** (CHF) — the point at which coalescing vapour blankets the surface, the liquid can no longer rewet it, and the wall temperature excursion destroys the device. The second is the effect of **bulk turbulence** on bubble dynamics: turbulence changes the departure diameter, the residence time, and the coalescence statistics, and therefore changes the heat transfer coefficient, but the mechanism is not captured by any wall-superheat correlation. Both are questions about *resolved, three-dimensional, time-dependent interface dynamics coupled to a turbulent velocity field*. They are questions for direct numerical simulation.

1.2 Why direct numerical simulation

Direct numerical simulation (DNS) resolves every dynamically active scale of the flow without a turbulence model. In a single-phase channel flow this means resolving down to the Kolmogorov scale. In an interface-resolved multiphase flow it additionally means resolving the interface itself: its position, its curvature, and — for boiling — the thermal boundary layers on both of its sides, since it is the *jump* in heat flux across the interface that sets the vaporisation rate.

DNS of boiling is therefore expensive in a way that compounds. A wall-bounded turbulent channel at a friction Reynolds number of a few hundred already demands $\mathcal{O}(10^8)$ grid points. Adding an interface adds the requirement that the interfacial layer be resolved by several cells everywhere it goes, and adds a stiff surface-tension time-step restriction. Adding phase change adds a temperature field with a thin thermal boundary layer, and a source term that couples all four equations together. The computational budget is only reachable on GPU-accelerated HPC systems, and only with a numerical method whose cost does not grow with the geometric complexity of the interface.

That last constraint is the decisive one, and it is the reason this project uses a phase-field method rather than volume-of-fluid or front tracking. It is developed in detail in Sections 3 and 5.

1.3 Why phase-field

A phase-field (or diffuse-interface) method replaces the sharp interface with a thin transition layer of finite thickness ε , across which an order parameter ϕ varies smoothly between the two bulk values. The interface is then *implicit*: it is wherever $\phi = 0.5$. Nothing needs to be reconstructed, no markers need to be advected, and no re-meshing is required when a bubble splits or two bubbles merge. Roccon (2025) states the resulting three advantages of the method compactly:

1. **Interface-blind cost.** “No geometrical reconstructions are required and thus the computational cost does not depend on the interface topology.” A simulation with 10,000 bubbles costs the same per grid point as one with a single bubble.
2. **Implementation simplicity.** The conservative Allen–Cahn equation is a second-order PDE — one advection term, one diffusion term, one sharpening term, one source term. It is a handful of stencil operations.
3. **A constant-coefficient pressure Poisson equation.** This is the property that makes the whole approach viable at scale, and it is worth stating precisely. In a variable-density projection method the pressure equation is normally $\nabla \cdot (\rho^{-1} \nabla p) = \text{rhs}$, a *variable-coefficient* elliptic problem that must be solved iteratively (multigrid, Krylov) at every time step. In the one-fluid momentum formulation used here, the pressure equation reduces to $\nabla^2 p = \text{rhs}$ with ∇^2 the plain Laplacian, which diagonalises under a Fourier transform and can therefore be solved *exactly, non-iteratively, in $\mathcal{O}(N \log N)$ operations by FFT*.

The third point is not a matter of convenience. Iterative elliptic solvers are the component of an incompressible flow solver that scales worst on distributed GPUs, because they require global reductions inside an inner loop. An FFT-based direct solve requires only pencil transposes, and libraries such as `cuFFT` and `cuDecomp` handle these efficiently to thousands of GPUs. The constant-coefficient property is what makes the method portable to the hardware on which a boiling DNS could actually be run. Everything in `src/pressure.py` exists to exploit it.

1.4 This project in context

The repository documented here — `boiling-phasefield-3d` — is my development and validation environment for the PhD project “*Phase-Field Modelling of Boiling Heat Transfer in Turbulent Flows: Extension to 2D/3D Direct Numerical Simulation*” at the Multiphase Flow Laboratory, University of Udine, under the supervision of Dr Alessio Roccon.

The scientific starting point is Roccon (2025), *Boiling heat transfer by phase-field method*, *Acta Mechanica* **236**, 5623–5638. That paper proposes and validates the method in 1D and 2D. It closes by naming the two things it did not do: “Future works will focus on more complex scenarios, as for instance film boiling and the parallelization and porting to GPUs of the proposed method.” The PhD project is precisely that continuation — extension to fully three-dimensional DNS in wall-bounded turbulence, and the port to a production HPC framework.

The production framework will be **FLOW36** (Roccon, Soligo and Soldati, 2025), a pseudo-spectral Fortran solver for phase-field multiphase turbulence that uses Fourier series in the periodic directions and Chebyshev polynomials in the wall-normal direction, parallelised with MPI and accelerated with OpenACC and `cuFFT`. Its sibling **MHIT36** (Roccon et al., 2025) is a finite-difference, multi-GPU code for multiphase homogeneous isotropic turbulence which uses the conservative diffuse-interface phase-field approach and an FFT-based Poisson solve, and which has been demonstrated from 128^3 up to 4096^3 on 1024 GPUs. Critically, **neither code currently contains**

boiling physics: both solve Navier–Stokes coupled to a phase-field equation, but neither carries an energy equation with a latent-heat source or a vaporisation-rate closure. Closing that gap is the substance of the PhD.

The Python prototype in this repository sits deliberately upstream of all of that. It is not a production solver and does not try to be. Its purpose is to let me implement every equation, every discretisation, and every coupling from Roccon (2025) in a language where I can read the whole solver in an afternoon, run a benchmark on a laptop in thirty seconds, and see immediately where the physics is right and where it is wrong — *before* committing any of it to Fortran, where a subtle sign error costs a week. Several of the findings reported in Sections 10 and 12 of this document are exactly the kind of error that this prototype exists to catch cheaply.

1.5 What this document is

This document is my own technical reference for the project. It has four jobs:

- to state the governing equations completely and derive them from first principles, with an exact cross-reference to the equation numbering of Roccon (2025), since the code comments reference that numbering throughout;
- to explain the numerical method, in particular *why* the one-fluid formulation yields a constant-coefficient Poisson equation, which is the load-bearing property of the whole approach;
- to walk through the actual implementation function by function, so that each line of code is tied back to an equation; and
- to record, honestly and quantitatively, what the prototype currently gets right, what it gets wrong, and exactly why — including the results of grid-refinement and diagnostic studies that I ran while writing this document, which are reported in Sections 10 and 11 and which identified several concrete defects listed in Section 12.

A note on sources. I was able to obtain the full text of Roccon (2025) (it is published open access under CC-BY), so every equation number, parameter value, and benchmark setup cited in this document has been checked against the paper itself rather than inferred from the code comments. Where the code and the paper disagree, I say so explicitly.

2 The Problem: Multiphase Flow with Phase Change

Before introducing any particular numerical method it is worth being precise about what makes this problem hard, because each difficulty maps onto a specific design decision later.

2.1 The sharp-interface statement of the problem

In the classical sharp-interface description, the domain Ω is partitioned at every instant into a liquid subdomain Ω_l and a vapour subdomain Ω_v , separated by a surface $\Gamma(t)$ of zero thickness. Within each subdomain the fluid obeys the incompressible Navier–Stokes and energy equations with constant properties:

$$\nabla \cdot \mathbf{u} = 0, \quad (1)$$

$$\rho_k \left(\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} \right) = -\nabla p + \mu_k \nabla^2 \mathbf{u} + \rho_k \mathbf{g}, \quad (2)$$

$$\frac{\partial T}{\partial t} + \mathbf{u} \cdot \nabla T = \alpha_k \nabla^2 T, \quad (3)$$

for $k \in \{l, v\}$. All the physics of boiling lives in the conditions imposed *on* Γ . Writing $\dot{m}$ for the mass flux per unit interfacial area ($\text{kg m}^{-2} \text{s}^{-1}$), $\mathbf{n}$ for the unit normal pointing from liquid into vapour, $\llbracket q \rrbracket = q_v - q_l$ for the jump across the interface, and κ for the mean curvature taken positive when the interface is convex toward the vapour, the jump conditions are:

$$\dot{m} = \rho_l(\mathbf{u}_l - \mathbf{u}_\Gamma) \cdot \mathbf{n} = \rho_v(\mathbf{u}_v - \mathbf{u}_\Gamma) \cdot \mathbf{n} \quad (\text{mass}) \quad (4)$$

$$\dot{m}^2 \left(\frac{1}{\rho_v} - \frac{1}{\rho_l} \right) + \llbracket p \rrbracket - \llbracket 2\mu \mathbf{n} \cdot \nabla \mathbf{u} \cdot \mathbf{n} \rrbracket = \sigma \kappa \quad (\text{normal momentum}) \quad (5)$$

$$\dot{m} h_{lv} = (k_v \nabla T_v - k_l \nabla T_l) \cdot \mathbf{n} \quad (\text{energy}) \quad (6)$$

$$T_\Gamma = T_{\text{sat}} \quad (\text{thermal equilibrium}) \quad (7)$$

In (5) the leading term is the **vapour recoil**: the momentum flux carried by the mass crossing the interface, obtained by multiplying $\dot{m}$ by the velocity jump (8). It vanishes when $\dot{m} = 0$, recovering the ordinary Young–Laplace balance of an isothermal two-phase flow. It is not represented in the present code, and Section 9.5 shows the closely related omission in the momentum advection term.

Equation (6) is the **Rankine–Hugoniot condition**: the net heat flux arriving at the interface from the two sides is exactly the latent heat consumed by the mass that crosses it. It is the physical statement that makes boiling a *coupled* problem, and it reappears verbatim as Eq. (12) of Roccon (2025) and as `src/energy.py:mdot_from_heatflux_2d` in this repository.

2.2 What makes this numerically hard

(a) **The interface is a moving free boundary of unknown position.** $\Gamma(t)$ is not given; it is part of the solution, and its motion is driven by $\dot{m}$, which in turn depends on temperature gradients

evaluated *at* $\Gamma(t)$. This circularity is the central difficulty. On a fixed Eulerian grid, Γ generally does not align with cell faces, so both the jump conditions and the one-sided gradients in (6) must be reconstructed by interpolation.

(b) Phase change destroys the divergence-free constraint. In a single-phase incompressible flow, $\nabla \cdot \mathbf{u} = 0$ everywhere, and this is what makes the projection method work. With vaporisation, mass crosses the interface at a rate $\dot{m}$ while the density drops from ρ_l to ρ_v ; volume is therefore *created* at the interface. From (4), the velocity jump across the interface is

$$[[\mathbf{u}]] \cdot \mathbf{n} = (\mathbf{u}_v - \mathbf{u}_l) \cdot \mathbf{n} = \dot{m} \left(\frac{1}{\rho_v} - \frac{1}{\rho_l} \right). \quad (8)$$

For water at atmospheric pressure with $\rho_l/\rho_v \approx 1600$, this expansion is violent: a vaporisation rate of only $\dot{m} = 0.1 \text{ kg m}^{-2} \text{ s}^{-1}$ produces a velocity jump of order 0.1 m/s across a layer a few cells thick. The pressure solver must handle a right-hand side with a localised, non-zero divergence source, and the momentum equation must not generate spurious currents from it.

(c) Large property ratios. Water/steam at atmospheric conditions gives $\rho_l/\rho_v \approx 1600$, $\mu_l/\mu_v \approx 23$, $k_l/k_v \approx 28$. A numerical scheme that is stable at ratio 1 can be violently unstable at ratio 1000, because any overshoot of the order parameter outside $[0, 1]$ produces a *negative density* when the mixture rule $\rho(\phi) = \rho_v\phi + \rho_l(1-\phi)$ is applied. This is precisely why Roccon selects the conservative Allen–Cahn formulation of Mirjalili et al. (2020), whose selling point is *provable boundedness* of ϕ . Section 4.5 returns to this.

(d) Surface tension is stiff. The capillary wave time scale on a grid of spacing Δx is

$$\Delta t_\sigma = \sqrt{\frac{(\rho_l + \rho_v) \Delta x^3}{2\pi\sigma}}, \quad (9)$$

the Brackbill–Kothe–Zemach restriction (Brackbill et al., 1992), which requires that the time step resolve the fastest capillary wave the grid can carry. Note that the constant in the denominator is quoted variously as 2π or π in the literature depending on the derivation; the 2π form is used throughout this document, and the distinction changes the result only by a factor $\sqrt{2}$. This is an *explicit-scheme* restriction, and it becomes progressively more severe as the grid is refined, scaling as $\Delta x^{3/2}$. Section 12.2 evaluates it for the parameters used in this repository and explains why the bubble benchmark is run with $\sigma = 0$.

(e) Latent heat is a large, localised source. The source term in the energy equation scales as h_{lv}/C_p . For water that ratio is $2.25 \times 10^6 / 4216 \approx 534 \text{ K}$ — i.e. the latent heat released or absorbed per unit mass corresponds to a temperature swing of 534 K, deposited into a layer only a few cells thick. Any error in *where* that source is deposited is amplified enormously.

(f) Topology changes are generic. Bubbles nucleate, grow, detach, rise, collide, coalesce, and break up. In nucleate boiling near CHF, thousands of bubbles interact simultaneously. Any method that requires explicit surgery on the interface representation when topology changes will either fail or become prohibitively expensive.

Difficulty (f) is what eliminates front tracking for this application; (c) is what selects conservative Allen–Cahn over Cahn–Hilliard; (b) and (d) are what dictate the structure of the time-stepping scheme; and (a) is what the vaporisation-rate closure of Section 7.5 exists to resolve.

3 Terminology and Foundations

3.1 Sharp interface versus diffuse interface

The sharp-interface picture of Section 2.1 is a *model*, not a physical fact. Real liquid–vapour interfaces have a finite thickness of order a few molecular diameters (nanometres, far from the critical point), across which density varies continuously. The sharp-interface description is the limit of that reality when the interface thickness is negligible compared to every other length in the problem — which, for a millimetre-scale bubble, it certainly is.

The diffuse-interface description takes the opposite tack: it retains a finite interface thickness, but treats it as a *numerical* parameter ε chosen for resolvability rather than as the true physical thickness. Instead of nanometres, ε is set to $\mathcal{O}(\Delta x)$ — in this repository, $\varepsilon = 1.5\Delta x$ by default. The justification is asymptotic: as $\varepsilon \rightarrow 0$, the diffuse-interface model recovers the sharp-interface model with the correct jump conditions (Section 4.4). One accepts a controlled $\mathcal{O}(\varepsilon)$ modelling error in exchange for a formulation in which every field is smooth and differentiable everywhere, and the interface never has to be located explicitly.

The trade-off is worth stating baldly, since it recurs throughout this document:

	Sharp interface	Diffuse interface
Interface	Explicit surface $\Gamma(t)$	Level set $\phi = 0.5$ of a smooth field
Jump conditions	Imposed as boundary conditions	Emerge from smoothly varying source terms
Topology change	Requires explicit surgery	Automatic
Field regularity	Discontinuous at Γ	C^∞ everywhere
Resolution needed	Interface can be sub-grid	Interface must span $\gtrsim 3$ cells
Error source	Reconstruction / interpolation	Finite ε , spurious currents

3.2 The order parameter

Throughout this repository the order parameter is

$$\phi(\mathbf{x}, t) \in [0, 1], \quad \phi = \begin{cases} 1 & \text{in the vapour,} \\ 0 & \text{in the liquid,} \end{cases} \quad (10)$$

with a smooth monotone transition in between. This convention — $[0, 1]$ rather than $[-1, +1]$ — is the one used by Roccon (2025) and by the conservative Allen–Cahn literature generally (Chiu and Lin, 2011; Mirjalili et al., 2020; Jain, 2022), and it is convenient because ϕ then acts directly as a *volume fraction*, so mixture properties are simple linear interpolations (Section 7.3).

The equilibrium profile of ϕ across a flat interface is a hyperbolic tangent,

$$\phi_{\text{eq}}(s) = \frac{1}{2} \left[1 - \tanh\left(\frac{s}{2\varepsilon}\right) \right], \quad (11)$$

where s is the signed normal distance from the $\phi = 0.5$ surface. This is exactly the profile used to initialise both benchmarks — see `examples/bubble_2d.py`:

```
r = np.sqrt((X - p.Lx/2)**2 + (Y - p.Ly/2)**2)
phi0 = 0.5 * (1 - np.tanh((r - R0) / (2 * p.eps)))
```

Two properties of (11) are used repeatedly and are worth deriving now. First, differentiating and using $d \tanh(z)/dz = 1 - \tanh^2 z$, together with $\tanh(s/2\varepsilon) = 1 - 2\phi$:

$$\frac{d\phi_{\text{eq}}}{ds} = -\frac{1}{4\varepsilon} [1 - (1 - 2\phi)^2] = -\frac{\phi(1 - \phi)}{\varepsilon}, \quad (12)$$

so that

$$|\nabla\phi| = \frac{\phi(1 - \phi)}{\varepsilon}. \quad (13)$$

This identity is the reason the whole method works without geometric reconstruction: it converts a *surface* quantity into a *volumetric* one analytically. It appears as Eq. (2) of Roccon (2025) and is implemented literally in `src/phase_field.py::mdot_volumetric`. Second, integrating (13) across the interface,

$$\int_{-\infty}^{\infty} \frac{\phi(1 - \phi)}{\varepsilon} ds = \int_0^1 d\phi = 1, \quad (14)$$

so $|\nabla\phi|$ is a properly normalised surface delta function, $\delta_{\Gamma} \approx |\nabla\phi|$. This is what licenses the continuum surface force model of Section 7.4.

3.3 Interface thickness and its resolution

Two constraints pull ε in opposite directions:

- **Accuracy** wants ε small, since the diffuse-interface model converges to the sharp-interface model as $\varepsilon \rightarrow 0$, and curvature-driven artefacts scale with ε .
- **Resolvability** wants ε large enough that the tanh profile is resolved by the grid. If the transition spans fewer than about three cells, the central-difference operators cannot represent it and the interface becomes a staircase.

The standard compromise is $\varepsilon = \mathcal{O}(\Delta x)$. Roccon (2025) uses $\varepsilon = 1.5\Delta x$ for the Stefan and adsorption benchmarks and $\varepsilon = \Delta x$ for the bubble benchmark. Jain (2022) gives the sharper criterion $\varepsilon > 0.5\Delta x$, with $\varepsilon \approx 0.5\Delta x$ preferred “because this keeps the interface, which is resolved on the grid, as sharp as possible.” This repository defaults to $\varepsilon = 1.5\Delta x$, set in `src/params.py::SimParams.__post_init__`:

```
def __post_init__(self):
    if self.eps is None:
        self.eps = 1.5 * self.dx
    if self.gamma is None:
        self.gamma = self.eps
```

The second line of that method is a defect, and Section 9.1 explains why.

A consequence that matters for interpreting results: because $\varepsilon \propto \Delta x$, refining the grid *simultaneously* refines the physical model. Convergence studies in a diffuse-interface method therefore measure a combination of discretisation error and modelling error. The measured second-order convergence reported in Section 11.1 should be read in that light.

3.4 A survey of interface-capturing and interface-tracking methods

Roccon (2025) divides the field into two families: methods that *track* the interface with explicit Lagrangian objects, and methods that *capture* it as an iso-level of a field. The four principal approaches are summarised below; Section 5 treats each in more depth in the specific context of phase change.

Front tracking (Unverdi and Tryggvason, 1992; Tryggvason et al., 2011) represents Γ by an unstructured mesh of connected Lagrangian marker points advected with the flow, superposed on a fixed Eulerian grid for the flow field. It is the most accurate representation of the interface — curvature and normals are computed directly from the surface mesh, and the jump conditions can be imposed sharply. Juric and Tryggvason (1998) used exactly this approach for the first full boiling computations. The fatal weakness for the present application is topology: coalescence and breakup require explicit mesh surgery, which is difficult, expensive, and does not parallelise cleanly.

Volume of fluid (VOF) (Hirt and Nichols, 1981) advects the liquid volume fraction $C \in [0, 1]$ in each cell. It is exactly mass-conservative by construction, which is its principal virtue, and it is the workhorse of industrial CFD. Its weakness is that C is discontinuous, so the interface must be geometrically reconstructed (PLIC) in each cell before it can be advected, and curvature — a second derivative of a reconstructed piecewise-linear surface — is notoriously noisy. Welch and Wilson (2000) developed the canonical VOF phase-change method.

Level set (Osher and Sethian, 1988) represents the interface as the zero level of a signed distance function ψ , advected by $\partial_t \psi + \mathbf{u} \cdot \nabla \psi = 0$. Because ψ is smooth, normals ($\nabla \psi / |\nabla \psi|$) and curvature ($\nabla \cdot (\nabla \psi / |\nabla \psi|)$) are accurate and cheap, and topology change is automatic. Its weakness is mass conservation: advection destroys the signed-distance property, requiring periodic reinitialisation, and each reinitialisation leaks mass. Son and Dhir (1998) and Gibou et al. (2007) applied level sets to boiling.

Phase field replaces the geometric field with a thermodynamically motivated order parameter whose evolution is derived from a free-energy functional. Topology change is automatic, normals and curvature are smooth, and — in the conservative Allen–Cahn variant — mass is conserved discretely without reinitialisation. This is the choice made here, and Section 4 derives it from first principles.

The essential comparison for this project:

Method	Topology change	Mass conservation	Curvature quality	Cost vs. topology
Front tracking	Manual surgery	Excellent	Excellent	Grows with complexity
VOF	Automatic	Exact	Poor (needs smoothing)	PLIC cost per cell
Level set	Automatic	Leaks; needs reinit.	Excellent	Reinitialisation cost

Method	Topology change	Mass conservation	Curvature quality	Cost vs. topology
Conservative Allen–Cahn	Automatic	Discretely conservative	Good	Independent

The last column is the one that decides the matter for turbulent boiling DNS.

4 Mathematical Foundations of Phase-Field Methods

This section derives the phase-field equations from thermodynamics, so that the terms appearing in the code are not arbitrary. The physical content originates with Cahn and Hilliard (1958) and Allen and Cahn (1979); its application to two-phase hydrodynamics is due principally to Jacqmin (1999).

4.1 The Ginzburg–Landau free energy functional

Cahn and Hilliard (1958) posed the question: what is the free energy of a system whose composition varies in space? Their answer, obtained by a gradient expansion of the local free energy density about the homogeneous state, is that to leading order the total free energy of a non-uniform system is

$$\mathcal{F}[\phi] = \int_{\Omega} \left[f_0(\phi) + \frac{\lambda}{2} |\nabla \phi|^2 \right] dV. \quad (15)$$

The two terms encode two competing physical tendencies:

- $f_0(\phi)$, the **bulk free energy density**, is minimised when the system is entirely in one pure phase or the other. It is written as a **double-well potential** with minima at $\phi = 0$ and $\phi = 1$:

$$f_0(\phi) = \beta \phi^2 (1 - \phi)^2. \quad (16)$$

Any intermediate value is energetically penalised. This is what drives phase *separation* and keeps the bulk phases pure.

- $\frac{\lambda}{2} |\nabla \phi|^2$, the **gradient energy**, penalises rapid spatial variation. This is what prevents the interface from collapsing to zero thickness and gives it a finite width.

The equilibrium interface profile is the one that balances these. Minimising (15) over profiles $\phi(s)$ connecting $\phi(-\infty) = 1$ to $\phi(+\infty) = 0$ gives the Euler–Lagrange equation $\lambda \phi'' = f'_0(\phi)$, whose first integral is $\frac{\lambda}{2} (\phi')^2 = f_0(\phi)$, i.e.

$$\frac{d\phi}{ds} = -\sqrt{\frac{2\beta}{\lambda}} \phi(1 - \phi). \quad (17)$$

Comparing with the tanh profile (11), whose derivative is $d\phi/ds = -\phi(1 - \phi)/\varepsilon$ by (13), this separable ODE integrates to exactly that profile provided the interface thickness is

$$\varepsilon = \sqrt{\frac{\lambda}{2\beta}}. \quad (18)$$

So the tanh profile is not an ansatz: it is the exact energy-minimising profile of the Ginzburg–Landau functional, and this is why the sharpening term in the conservative Allen–Cahn equation drives ϕ back toward it. The excess free energy stored in the profile, integrated across the interface, is the **surface tension**:

$$\sigma = \int_{-\infty}^{\infty} \lambda \left(\frac{d\phi}{ds} \right)^2 ds = \frac{\lambda}{6\varepsilon} = \frac{1}{6} \sqrt{2\lambda\beta}. \quad (19)$$

The middle step uses the substitution $ds = -\varepsilon d\phi/[\phi(1-\phi)]$, which converts the integral to $(1/\varepsilon) \int_0^1 \phi(1-\phi) d\phi = 1/(6\varepsilon)$; the final step substitutes (18). Equation (19) is what connects the model parameters λ, β to a measurable physical property, and the factor $1/6$ appearing here is the same one that reappears — inverted — as the factor of 6 in the CSF force of Section 7.4.

4.2 The chemical potential and the variational derivative

The thermodynamic driving force for evolution of ϕ is the **chemical potential**, defined as the variational derivative of the free energy:

$$\mu_\phi \equiv \frac{\delta \mathcal{F}}{\delta \phi} = \frac{\partial f_0}{\partial \phi} - \nabla \cdot (\lambda \nabla \phi) = f'_0(\phi) - \lambda \nabla^2 \phi. \quad (20)$$

The derivation is the standard one: perturb $\phi \rightarrow \phi + \delta\phi$ in (15),

$$\delta \mathcal{F} = \int_{\Omega} [f'_0(\phi) \delta\phi + \lambda \nabla \phi \cdot \nabla \delta\phi] dV,$$

then integrate the second term by parts, discarding the boundary term (which vanishes for periodic or no-flux boundaries), to obtain $\delta \mathcal{F} = \int_{\Omega} [f'_0(\phi) - \lambda \nabla^2 \phi] \delta\phi dV$. Equilibrium is $\mu_\phi = 0$; away from equilibrium, $-\mu_\phi$ is the thermodynamic force driving relaxation.

Two distinct relaxation dynamics follow from (20), depending on whether ϕ is a conserved quantity.

4.3 Cahn–Hilliard versus Allen–Cahn

Cahn–Hilliard (conserved dynamics). If ϕ is a conserved order parameter — a concentration, which cannot be created or destroyed, only transported — then its evolution must take the form of a conservation law $\partial_t \phi = -\nabla \cdot \mathbf{J}$ with a flux driven down the chemical potential gradient, $\mathbf{J} = -M \nabla \mu_\phi$. This gives the **Cahn–Hilliard equation**:

$$\frac{\partial \phi}{\partial t} + \mathbf{u} \cdot \nabla \phi = \nabla \cdot (M \nabla \mu_\phi) = M \nabla^2 [f'_0(\phi) - \lambda \nabla^2 \phi]. \quad (21)$$

Mass is conserved exactly, because the right-hand side is a pure divergence. The price is that (21) is **fourth order in space** — the $\nabla^2 \nabla^2 \phi$ term. That is expensive: it requires a wide stencil, and explicit time integration is subject to a punishing $\Delta t \sim \Delta x^4$ stability restriction. It also exhibits a well-known artefact, *Ostwald ripening*: small drops spontaneously shrink and disappear in favour of large ones, driven by the curvature-dependence of μ_ϕ , which is physical for a phase-separating alloy but is a spurious mass sink for a spray of droplets. Jacqmin (1999) built the first systematic two-phase Navier–Stokes solver on the Cahn–Hilliard equation, and it remains widely used, but the fourth-order term is a genuine barrier at DNS scale.

Allen–Cahn (non-conserved dynamics). If ϕ is *not* conserved — an order parameter labelling a crystallographic variant, in the original setting of Allen and Cahn (1979) — then relaxation is simply proportional to the driving force:

$$\frac{\partial \phi}{\partial t} = -M \mu_\phi = -M [f'_0(\phi) - \lambda \nabla^2 \phi]. \quad (22)$$

This is only **second order in space**, hence cheap and easy. Allen and Cahn’s celebrated result is that the interface defined by (22) moves by mean curvature: the normal velocity is $v_n = -M\lambda\kappa$. That is exactly the problem for two-phase flow. A spherical bubble under (22) shrinks and vanishes even in a quiescent fluid, purely as a numerical artefact. Applied naively to a two-phase flow, the standard Allen–Cahn equation does not conserve mass at all.

The dilemma is therefore: Cahn–Hilliard conserves mass but is fourth order and ripens; Allen–Cahn is second order but does not conserve mass and shrinks bubbles by curvature flow. The conservative Allen–Cahn formulation resolves it.

4.4 The sharp-interface limit

Why should any of this reproduce the sharp-interface jump conditions of Section 2.1? The answer is matched asymptotic expansions in the small parameter ε , and although the full analysis is long, the structure of the argument is worth recording because it is what justifies the entire modelling approach.

One introduces two length scales: an *outer* scale, on which distances are measured in units of the macroscopic length L , and an *inner* scale, on which the normal coordinate is stretched as $\eta = s/\varepsilon$. In the outer region, expanding $\phi = \phi_0 + \varepsilon\phi_1 + \mathcal{O}(\varepsilon^2)$ and taking $\varepsilon \rightarrow 0$ in the phase-field equation forces $f'_0(\phi_0) = 0$ at leading order, i.e. $\phi_0 \in \{0, 1\}$: the outer solution is exactly the pure bulk phases, and the governing equations reduce to the single-phase Navier–Stokes and energy equations with constant properties, as required.

In the inner region, rewriting the operators in stretched coordinates ($\nabla \rightarrow \varepsilon^{-1}\mathbf{n}\partial_\eta + \nabla_\Gamma$, $\nabla^2 \rightarrow \varepsilon^{-2}\partial_{\eta\eta} + \varepsilon^{-1}\kappa\partial_\eta + \dots$) and collecting powers of ε : the $\mathcal{O}(\varepsilon^{-2})$ balance recovers the equilibrium tanh profile (11); the $\mathcal{O}(\varepsilon^{-1})$ balance yields a solvability condition, which is precisely the interfacial jump condition. Matching the inner and outer expansions in the overlap region then shows that the outer problem sees exactly the sharp-interface boundary conditions (4)–(7), with an error of $\mathcal{O}(\varepsilon)$.

The practical consequences are the ones to remember:

- The diffuse-interface model is *consistent*: it converges to the right sharp-interface problem as $\varepsilon \rightarrow 0$.
- The leading error is $\mathcal{O}(\varepsilon)$ in general and $\mathcal{O}(\varepsilon^2)$ for well-designed formulations; since $\varepsilon \propto \Delta x$ in practice, this contributes directly to the observed convergence order. Section 11.1 measures approximately second-order convergence for the bubble benchmark, consistent with the latter.
- Curvature-related artefacts, including spurious currents, scale with ε and do not vanish at fixed ε no matter how the flow solver is refined.

4.5 The conservative Allen–Cahn equation

The resolution of the Cahn–Hilliard/Allen–Cahn dilemma came from an unexpected direction: the *conservative level set* method of Olsson and Kreiss (2005), which advects a hyperbolic-tangent-shaped field and then re-sharpens it by a compressive flux. Chiu and Lin (2011) observed that the advection step and the re-sharpening step could be merged into a single PDE, producing a formulation that “can preserve the total mass as the Cahn–Hilliard equation, but the calculation

and implementation are much simpler.” Mirjalili, Ivey and Mani (2020) then established the property that makes it usable at high density ratio: with central differences and appropriately chosen parameters, ϕ is *provably bounded* in $[0, 1]$.

The resulting **conservative Allen–Cahn (CAC)** equation, in the form used by Roccon (2025) and implemented in this repository, is

$$\frac{\partial \phi}{\partial t} + \nabla \cdot (\mathbf{u}\phi) = \nabla \cdot \left[\gamma \left(\varepsilon \nabla \phi - \phi(1 - \phi) \frac{\nabla \phi}{|\nabla \phi|} \right) \right]. \quad (23)$$

Read the right-hand side as a single flux $\mathbf{J} = \gamma(\varepsilon \nabla \phi - \phi(1 - \phi)\mathbf{n})$ with two competing parts:

- $\gamma \varepsilon \nabla \phi$ — a **diffusive (smoothing)** flux, which spreads the interface;
- $-\gamma \phi(1 - \phi)\mathbf{n}$ — a **sharpening (compressive)** flux directed along the interface normal $\mathbf{n} = \nabla \phi / |\nabla \phi|$, which compresses it.

At the equilibrium tanh profile, identity (13) gives $\phi(1 - \phi) = \varepsilon |\nabla \phi|$, so $\phi(1 - \phi)\mathbf{n} = \varepsilon |\nabla \phi| \nabla \phi / |\nabla \phi| = \varepsilon \nabla \phi$, and the two fluxes cancel **exactly**: $\mathbf{J} = \mathbf{0}$. The tanh profile is a stationary state of the right-hand side. Any deviation from it — smearing by advection, steepening by strain — produces a non-zero net flux that restores it. This is the entire mechanism, and it is what replaces the reinitialisation step of a level-set method.

Three properties follow immediately, and together they explain why this formulation was chosen:

1. **Discrete conservation.** The right-hand side is a pure divergence. Integrating (23) over a periodic domain gives $d/dt \int \phi dV = 0$ exactly (absent a source), and the same holds discretely because the discrete divergence of the discrete flux telescopes. No reinitialisation, no mass redistribution, no leak.
2. **Second order in space.** Unlike Cahn–Hilliard, there is no biharmonic term. Stencils are narrow, the explicit time-step restriction is $\Delta t \sim \Delta x^2$ rather than Δx^4 , and — importantly for the target architecture — the stencil maps cleanly onto a GPU halo exchange.
3. **Provable boundedness.** Mirjalili et al. (2020) and Jain et al. (2020) give the criterion for ϕ to remain in $[0, 1]$ with central differences:

$$\Gamma^* \geq \frac{1}{2\epsilon^* - 1}, \quad \Gamma^* = \frac{\gamma}{|\mathbf{u}|_{\max}}, \quad \epsilon^* = \frac{\varepsilon}{\Delta x}. \quad (24)$$

Jain (2022) states the practical form of this as $\gamma \geq |\mathbf{u}|_{\max}$ and $\varepsilon > 0.5\Delta x$.

Criterion (24) deserves emphasis because it is violated in this repository. γ is a **velocity-scale parameter**, with units of m/s — Jain (2022) calls it exactly that (“ Γ represents the velocity-scale parameter”). Dimensional analysis confirms it: the right-hand side of (23) has dimensions $[\gamma][\varepsilon][\phi]/L^2 = [\gamma]/L$, which must match $[\phi]/T = 1/T$, forcing $[\gamma] = L/T$. The default `gamma = eps` in `src/params.py` therefore assigns a *length* to a *velocity*. Section 9.1 quantifies the consequences.

Finally, the comparison that motivates the whole choice:

	Cahn–Hilliard	Standard Allen–Cahn	Conservative Allen–Cahn
Spatial order	4th	2nd	2nd
Mass conservation	Exact	None	Exact (discrete)

	Cahn–Hilliard	Standard Allen–Cahn	Conservative Allen–Cahn
Curvature-driven shrinkage	Ostwald ripening	Yes (severe)	No
Explicit Δt limit	$\sim \Delta x^4$	$\sim \Delta x^2$	$\sim \Delta x^2$
Boundedness of ϕ	Not guaranteed	Not guaranteed	Provable
Reinitialisation needed	No	No	No

The last two rows are what make it viable at $\rho_l/\rho_v = 1000$, where an excursion of ϕ below zero means a negative density and instant divergence.

5 Classical Approaches to Boiling Simulation

This section surveys how the three older families of methods handle phase change, since they define the state of the art against which Roccon (2025) validates, and since several of them supply the reference data used in Section 10.

5.1 Front tracking

Unverdi and Tryggvason (1992) introduced the modern front-tracking method: a fixed Eulerian grid for the flow, plus an unstructured Lagrangian surface mesh for the interface, with information passed between them by smoothed delta functions. Juric and Tryggvason (1998) extended it to boiling in what remains a landmark paper. Their formulation is a **single-field** one — one set of mass, momentum and energy equations for the whole domain, with interfacial effects (surface tension, interphase mass transfer, latent heat) entering as delta-function source terms located on the tracked front. Structurally this is the same “one-fluid” philosophy adopted here; the difference is purely in how the interface is represented.

Esmaeeli and Tryggvason (2004) applied it to film boiling, and Irfan and Muradoglu (2017) developed a front-tracking evaporation method whose adsorption benchmark Roccon (2025) reproduces in his Section 3.3.

Strengths for phase change: the interface normal $\mathbf{n}$ and curvature κ are computed directly from the surface mesh and are very accurate, and — crucially — the one-sided temperature gradients ∇T_v and ∇T_l required by the Rankine–Hugoniot condition (6) can be evaluated by probing at known offsets from a *known* surface. The probe method of Udaykumar et al. (1996), which Roccon adopts, originates here.

Weaknesses: topology change requires manual surgery; the Lagrangian mesh must be adaptively refined and coarsened as it stretches; and the whole structure parallelises badly, since the surface mesh does not decompose along the same lines as the volume grid.

5.2 Volume of fluid

Welch and Wilson (2000) produced the canonical VOF phase-change method. The liquid volume fraction C obeys

$$\frac{\partial C}{\partial t} + \nabla \cdot (\mathbf{u}C) = C \nabla \cdot \mathbf{u} - \frac{\dot{m}'''}{\rho_l}, \quad (25)$$

with the interface geometrically reconstructed each step (PLIC) so that the flux across each face can be computed exactly. The vaporisation rate comes from a heat-flux balance equivalent to (6), evaluated on the reconstructed interface.

Strengths: exact mass conservation, maturity, and availability in every major industrial code. Sun et al. (2014) used a VOF approach in Fluent to generate the Stefan-problem reference data against which Roccon (2025) validates in his Section 3.2; Kunkelmann and Stephan (2009) and Giustini and Issa (2021) provide further VOF reference data used in his Section 3.3.

Weaknesses for phase change: curvature from a reconstructed PLIC surface is noisy, which pollutes surface tension; and the reconstruction cost per cell is real, and rises with interfacial complexity. Kharangate and Mudawar (2017) survey the resulting spread in published boiling results.

5.3 Level set

Osher and Sethian (1988) introduced level sets; Son and Dhir (1998) applied them to film boiling near critical pressure. Gibou et al. (2007) developed a *sharp-interface* level-set method for phase change in which the jump conditions are imposed exactly using ghost-fluid extrapolation, and Tanguy et al. (2007; 2014) developed level-set vaporisation methods and the two-dimensional bubble growth benchmark that Roccon reproduces in his Section 3.4 — and that this repository reproduces in `examples/bubble_2d.py`.

Strengths: accurate normals and curvature from the smooth signed-distance field; automatic topology change; and the ghost-fluid machinery makes sharp imposition of the Rankine–Hugoniot condition natural.

Weaknesses: mass conservation. Advection destroys the signed-distance property, requiring reinitialisation, and reinitialisation moves the zero level slightly. Over a long boiling simulation, that becomes a systematic mass drift — which is fatal when the quantity of interest *is* the vaporised mass. Hybrid CLSVOF methods exist to patch this at the cost of carrying two fields.

One specific point from Tanguy et al. (2014) is worth recording because Roccon (2025) singles it out: their “delta formulation” (a one-fluid approach with smeared source terms) exhibits an *overshoot* in the bubble growth rate. Roccon reports that this overshoot does not appear in his phase-field results, and attributes it to the asymmetry of the phase-field profile and the smoothed velocity jump across the interface noted by Sun and Beckermann (2004), which suppresses the spurious currents that otherwise corrupt interface advection.

5.4 Phase field for phase change

Sun and Beckermann (2004) provided the theoretical foundation for diffuse-interface two-phase flow with density contrast, deriving the mass and momentum equations by averaging and — importantly for interpreting the results here — showing that the mixture velocity profile is *asymmetric* with respect to the phase-field profile, with most of the velocity jump occurring on the $\phi > 0.5$ side. Wang et al. (2021) developed a Cahn–Hilliard-based phase-field method for boiling; Haghani-Hassan-Abadi et al. (2021) and Tamura and Katono (2022) developed conservative Allen–Cahn-based phase-change models, and Mohammadi-Shad and Lee (2017) a phase-field lattice-Boltzmann boiling model with a sharp-interface energy solver.

Roccon (2025) sits in this last lineage, and its specific contribution is the combination: conservative Allen–Cahn (for boundedness at high density ratio) + one-fluid Navier–Stokes with a phase-change mass source (for the constant-coefficient Poisson equation) + probe-based Rankine–Hugoniot closure (for accuracy of the vaporisation rate).

6 Research Evolution

It is useful to lay out the intellectual lineage explicitly, because this project is one more step along a clearly-defined path and the next step is easier to see when the previous ones are laid out.

1958–1979: thermodynamic foundations. Cahn and Hilliard (1958) establish that the free energy of a non-uniform system contains a gradient term, giving rise to (15) and hence to a finite-thickness interface with a computable surface tension. Allen and Cahn (1979) derive the non-conserved relaxation dynamics and prove that its interfaces move by mean curvature. Neither paper is about fluid mechanics; both are about microstructure in alloys.

1999: phase field meets Navier–Stokes. Jacqmin (1999) couples the Cahn–Hilliard equation to the incompressible Navier–Stokes equations, showing that surface tension can be represented as a body force derived from the free-energy functional and that the resulting model reproduces the correct sharp-interface hydrodynamics. This is the origin of phase-field CFD.

2005–2011: the conservative reformulation. Olsson and Kreiss (2005) propose the conservative level set, advecting a tanh-shaped field and re-sharpening it with a compressive flux. Chiu and Lin (2011) merge advection and re-sharpening into a single second-order PDE — the conservative Allen–Cahn equation (23) — obtaining Cahn–Hilliard’s mass conservation at Allen–Cahn’s cost.

2020–2022: boundedness and accuracy. Mirjalili, Ivey and Mani (2020) prove that ϕ remains bounded in $[0, 1]$ under central differencing given criterion (24), which is what makes the method safe at density ratio 1000. Jain et al. (2020) extend the result to compressible flows; Jain (2022) proposes the accurate conservative diffuse-interface (ACDI) variant and gives the practical parameter guidance $\gamma \geq |\mathbf{u}|_{\max}$, $\varepsilon > 0.5\Delta x$.

2022–2023: conservative Allen–Cahn for turbulence at scale. Mangani et al. (2022) study density and viscosity effects on bubble deformation, breakage and coalescence in turbulence. Roccon, Zonta and Soldati (2023) give the definitive account of phase-field modelling for drop-laden turbulence — the direct non-boiling ancestor of this project, and the physics implemented in FLOW36. That paper explicitly discusses “how to model surface tension changes due to surfactant distribution and heat and mass transfer fluxes”, i.e. it identifies the extension that Roccon (2025) then carries out.

2025: phase change. Roccon (2025) adds the energy equation, the latent-heat source, the phase-change mass source in the continuity equation, and the Rankine–Hugoniot vaporisation closure, and validates against four benchmarks over density ratios from 1 down to 5×10^{-4} . In parallel, the production codes reach maturity: FLOW36 (Roccon, Soligo and Soldati, 2025) and MHIT36 (Roccon et al., 2025), the latter scaling to 4096^3 on 1024 GPUs.

This project. Roccon (2025) validates in 1D and 2D and closes by naming film boiling, parallelisation, and GPU porting as future work. The gap is therefore sharply defined: the *method* is validated, the *production codes* exist and scale, but the two have not been joined, and nothing has been done in 3D or in turbulence. The PhD project closes that gap. This repository is its first stage — an independent Python re-implementation of every equation in Roccon (2025), with `*_3d` variants of every operator already written, whose purpose is to make every algorithmic decision explicit and every error visible before the Fortran port begins.

7 The Governing Equation Set

This section states each of the four coupled equations as implemented, gives its physical derivation, and cross-references the exact equation number in Roccon (2025). All equation numbers of the form “(R.*n*)” refer to that paper.

7.1 Conservative Allen–Cahn with vaporisation source — (R.1), (R.2)

$$\boxed{\frac{\partial \phi}{\partial t} + \nabla \cdot (\mathbf{u}\phi) = \nabla \cdot \left[\gamma \left(\varepsilon \nabla \phi - \phi(1-\phi) \frac{\nabla \phi}{|\nabla \phi|} \right) \right] + \frac{\dot{m}'''}{\rho_v}} \quad (26)$$

This is Eq. (R.1). The left-hand side is conservative advection of ϕ by the mixture velocity. The first two right-hand-side terms are the diffusive and sharpening fluxes already analysed in Section 4.5; Roccon describes them as terms that “allow to preserve the interfacial profile during the computation, e.g. a hyperbolic tangent profile”, with strength tuned by γ .

The third term is what makes this a *boiling* equation. $\dot{m}'''$ is the vaporisation rate per unit volume, in $\text{kg m}^{-3} \text{s}^{-1}$ — “the amount of liquid that vaporizes in the unit time and per unit volume”. Dividing by ρ_v converts a mass rate into a *volume* rate of vapour production, which is exactly the rate at which ϕ (a volume fraction) must increase.

The conversion from the surface rate $\dot{m}$ ($\text{kg m}^{-2} \text{s}^{-1}$), which is what the physics of Section 7.5 delivers, to the volumetric rate is Eq. (R.2):

$$\boxed{\dot{m}''' = \dot{m} |\nabla \phi| \approx \dot{m} \frac{\phi(1-\phi)}{\varepsilon}} \quad (27)$$

using identity (13). This is the central trick of the whole diffuse-interface approach to phase change: the surface source is smeared over the interfacial layer by the normalised delta function $|\nabla \phi|$, in a way that (i) requires no knowledge of where the interface is, (ii) integrates to the correct total across the layer, and (iii) is a smooth function of ϕ and hence differentiable. Implementation: `src/phase_field.py::mdot_volumetric`.

7.2 Mass conservation with a phase-change source — (R.3)

$$\boxed{\nabla \cdot (\rho \mathbf{u}) = \dot{m}''' \left(1 - \frac{\rho_v}{\rho_l} \right)} \quad (28)$$

This is Eq. (R.3), and it is the equation that makes boiling structurally different from isothermal two-phase flow. The factor $(1 - \rho_v/\rho_l)$ is easy to get wrong, so it is worth verifying that (28) is exactly the diffuse-interface counterpart of the sharp-interface mass jump condition (4).

Consider a flat interface in one dimension, with the liquid at rest ($u_l = 0$) and $\mathbf{n}$ pointing from liquid into vapour. Integrating (28) across the interfacial layer and using $\dot{m}''' = \dot{m} |\nabla \phi|$ from (27) together with the normalisation $\int |\nabla \phi| ds = 1$ established in Section 3.2:

$$\llbracket \rho u \rrbracket = \int \dot{m} |\nabla \phi| \left(1 - \frac{\rho_v}{\rho_l} \right) ds = \dot{m} \left(1 - \frac{\rho_v}{\rho_l} \right). \quad (29)$$

Now compute the same jump from the sharp-interface condition (4). With u_Γ the interface velocity, that condition gives $u_v = u_\Gamma + \dot{m}/\rho_v$ and $u_l = u_\Gamma + \dot{m}/\rho_l$, so

$$[\![\rho u]\!] = \rho_v u_v - \rho_l u_l = (\rho_v u_\Gamma + \dot{m}) - (\rho_l u_\Gamma + \dot{m}) = (\rho_v - \rho_l) u_\Gamma. \quad (30)$$

Setting $u_l = 0$ gives $u_\Gamma = -\dot{m}/\rho_l$, hence $[\![\rho u]\!] = -(\rho_v - \rho_l)\dot{m}/\rho_l = \dot{m}(1 - \rho_v/\rho_l)$ — identical to the diffuse-interface result. Equation (28) is therefore the correct smeared form of the mass jump condition, with the interfacial delta function supplied by $|\nabla\phi|$.

Roccon’s own comment on this equation is the essential physical reading, and I reproduce it because it is the sentence that determines the structure of the pressure solver: *“only for the special case $\rho_v = \rho_l$ the flow is divergence-free in the entire domain while, in the most general case where $\rho_v \neq \rho_l$, the divergence-free property is lost in the interfacial region (assuming a non-zero vaporization rate).”*

Two consequences:

- The source is **non-zero only where** $\phi(1 - \phi) \neq 0$ — i.e. only inside the interfacial layer, since $\dot{m}''' \propto \phi(1 - \phi)$. In both bulk phases the flow is divergence-free as usual.
- The magnitude of the source is controlled by the density ratio. At $\rho_v = \rho_l$ it vanishes identically (which is why the Stefan benchmark in this repository, run with matched densities, has no flow at all and can omit the Navier–Stokes solve entirely).

Implementation: `src/flow.py::ns_step_2d`, as `mass_src = mdot_vol * (1.0 - p.rho_v / p.rho_l)`.

7.3 One-fluid Navier–Stokes — (R.4), with mixture properties (R.5), (R.6)

$$\boxed{\frac{\partial(\rho \mathbf{u})}{\partial t} + \nabla \cdot (\rho \mathbf{u} \mathbf{u}) = -\nabla p + \nabla \cdot [\mu (\nabla \mathbf{u} + \nabla \mathbf{u}^T)] + \mathbf{f}_\sigma} \quad (31)$$

This is Eq. (R.4), written in **conservative momentum form** with $\mathbf{w} = \rho \mathbf{u}$ as the primary variable. The one-fluid approach means one set of equations is solved over the entire domain; the distinction between phases enters only through the property fields, which are linear interpolants in ϕ — Eqs. (R.5) and (R.6):

$$\rho(\phi) = \rho_v \phi + \rho_l (1 - \phi), \quad \mu(\phi) = \mu_v \phi + \mu_l (1 - \phi). \quad (32)$$

The linearity here is not an approximation to something better; it is the correct volume-averaged density, given that ϕ is the vapour volume fraction. It also makes clear why boundedness of ϕ is non-negotiable: if ϕ overshoots to 1.001 at density ratio 1000, then $\rho = 1.001\rho_v - 0.001\rho_l = 1.001 - 1 = 0.001 \text{ kg/m}^3$ for the water parameters, a factor-of-1000 error; if it overshoots much further, ρ goes negative and the solver diverges within a step. Implementation: `src/flow.py::rho` and `src/flow.py::mu`.

The choice of conservative form (writing $\partial_t(\rho \mathbf{u}) + \nabla \cdot (\rho \mathbf{u} \mathbf{u})$ rather than $\rho(\partial_t \mathbf{u} + \mathbf{u} \cdot \nabla \mathbf{u})$) matters here in a way that it does not in single-phase flow. The two forms differ by $\mathbf{u}[\partial_t \rho + \nabla \cdot (\rho \mathbf{u})]$, which vanishes only when continuity holds in the homogeneous form. With phase change it does not vanish: it equals $\mathbf{u} \dot{m}'''(1 - \rho_v/\rho_l)$ by (28). Section 9.5 shows that the implementation in `src/flow.py` uses the non-conservative form and therefore omits this term.

7.4 Surface tension: the continuum surface force — (R.7), (R.8)

$$\boxed{\mathbf{f}_\sigma = 6 \sigma \kappa \phi(1 - \phi) \nabla \phi} \quad \text{with} \quad \kappa = \nabla \cdot \mathbf{n} = \nabla \cdot \left(\frac{\nabla \phi}{|\nabla \phi|} \right) \quad (33)$$

These are Eqs. (R.7) and (R.8). The idea, due to Brackbill, Kothe and Zemach (1992), is to replace the surface stress $\sigma \kappa \mathbf{n} \delta_\Gamma$ by an equivalent volumetric body force distributed across the interfacial layer, so that it can be added directly to the momentum equation without any interface reconstruction.

The structure of (33) follows from Section 3.2: $\sigma \kappa$ is the Young–Laplace pressure jump, $\nabla \phi$ supplies the direction (normal to the interface), and $\phi(1 - \phi)$ localises the force to the interfacial layer. The factor 6 is the normalisation constant for this particular localisation: since $\int \phi^2(1 - \phi)^2 ds/\varepsilon = 1/6$ for the tanh profile, the factor 6 ensures that integrating $\mathbf{f}_\sigma$ across the layer recovers exactly $\sigma \kappa \mathbf{n}$, so that the Young–Laplace law is reproduced in the sharp-interface limit. Roccon attributes this “localized continuous surface force” form to Mirjalili, Khanwale and Mani (2023). Implementation: `src/flow.py::surface_tension_2d` and `surface_tension_3d`, with curvature from `_curvature_2d/_curvature_3d`.

The known weakness of any CSF model is **spurious (parasitic) currents**: because the discrete curvature and the discrete pressure gradient are not exactly compatible, the computed $\mathbf{f}_\sigma$ is not exactly balanced by ∇p , and a residual force drives an unphysical vortical flow near the interface with magnitude $\sim \sigma/\mu$. Section 12.2 discusses the associated time-step restriction, which is the reason $\sigma = 0$ in the bubble benchmark.

7.5 Energy equation with latent heat — (R.9), (R.10), (R.11)

$$\boxed{\frac{\partial T}{\partial t} + \nabla \cdot (\mathbf{u}T) = \nabla \cdot (\alpha \nabla T) + S_t} \quad (34)$$

This is Eq. (R.9), obtained from the full energy equation by neglecting viscous dissipation and pressure work — both legitimate given incompressibility of the two phases and the low Mach numbers involved. The thermal diffusivity is a mixture property in the same sense as ρ and μ , Eq. (R.10):

$$\alpha(\phi) = \alpha_v \phi + \alpha_l (1 - \phi), \quad \alpha_k = \frac{k_k}{\rho_k C_{p,k}}. \quad (35)$$

Implementation: `src/energy.py::alpha_mix`. The latent-heat source is Eq. (R.11):

$$\boxed{S_t = -\frac{h_{lv}}{C_p} \left(\frac{\partial \phi}{\partial t} + \nabla \cdot (\mathbf{u}\phi) \right)} \quad (36)$$

The physical content is direct: $\partial_t \phi + \nabla \cdot (\mathbf{u}\phi)$ is the net rate of vapour creation per unit volume, each unit of which absorbs h_{lv} joules per kilogram; dividing by C_p converts that energy into a temperature rate. The minus sign encodes that vaporisation *cools*.

There is an important simplification here that the code follows and that must be understood to read `src/solver.py` correctly. Roccon (2025) argues that in a boiling problem one may assume “that one

phase (and thus the interface) is always at saturation temperature. This implies that vaporization is driven by one phase solely, i.e. a superheated liquid or vapor. Using this assumption, as the flow field and phase-field carry information related to the latent heat mechanism, **the source term S_t can be neglected**. Thus, it is sufficient to solve the energy equation only in the superheated phase (the one driving phase change) while the other phase is kept at constant and uniform saturation conditions.”

So the production algorithm is: advance T with advection and diffusion only, then overwrite $T = T_{\text{sat}}$ throughout the non-superheated phase. That overwrite is equivalent to imposing a Dirichlet condition $T = T_{\text{sat}}$ at the interface, which is the sharp-interface condition (7); the latent heat then enters the problem *only* through the vaporisation rate closure, not through a volumetric source. `src/solver.py::run_2d` implements the saturation overwrite. Section 9.6 examines how its treatment of S_t departs from (36).

7.6 Vaporisation rate from the Rankine–Hugoniot balance — (R.12)

$$\dot{m} = q_{v \rightarrow l} - q_{l \rightarrow v} = \frac{(k_v \nabla T_v - k_l \nabla T_l) \cdot \mathbf{n}}{h_{lv}} \quad (37)$$

This is Eq. (R.12), and it is the closure that makes the system self-consistent. The physical statement is a jump balance at the interface: whatever net heat arrives from the two sides and is not conducted onward must be consumed as latent heat by the mass that changes phase. It is the same relation as (6).

Roccon notes that this “energy (or heat conduction) model” class is one of two available; the other is the **kinetic** class based on the kinetic theory of gases, of which the Tanasawa model, Eq. (R.13),

$$\dot{m} = \frac{2\chi}{2 - \chi} \left(\frac{M}{2\pi R_g} \right)^{1/2} \frac{\rho_v h_{lv} (T - T_{\text{sat}})}{T_{\text{sat}}^{3/2}}, \quad (38)$$

is the most used. Kinetic models are simpler because they need only a local temperature, not one-sided gradients; Roccon nonetheless chooses the energy-based model (R.12), because in a phase-field method “accurate information of the interface normal are readily available at run time” as $\mathbf{n} = \nabla \phi / |\nabla \phi|$.

The probe method. Evaluating (37) correctly is the single most delicate part of the whole algorithm, because ∇T_v and ∇T_l are *one-sided* gradients evaluated on opposite sides of a surface that does not lie on grid points. A central difference straddling the interface does not compute either of them; it computes an average that is contaminated by the temperature discontinuity in gradient. Roccon’s Section 2.4 therefore adopts the probe method of Udaykumar et al. (1996), in three steps:

1. The exact position $\mathbf{x}_i$ of the $\phi = 0.5$ iso-contour is identified, using the signed-distance property of the phase-field variable (Jain, 2022).
2. The temperature gradients are evaluated at two **probe points** $\mathbf{x}_p = \mathbf{x}_i \pm \Delta \mathbf{n}$, one in the vapour and one in the liquid, with the offset Δ “of the order of the grid spacing”. These are obtained by interpolation from surrounding grid values, so each gradient samples only its own phase.
3. The resulting surface rate $\dot{m}$ is smeared into the interfacial layer using (27).

This repository does not implement step 1 or step 2. `src/energy.py::mdot_from_heatflux_2d` evaluates a single central-difference gradient at grid points and states so in its own docstring. The consequences are quantified in Sections 10.2 and 11.2, and the function additionally contains an outright error identified in Section 9.4. Implementing the full probe method is the highest-priority correction on the roadmap.

7.7 Summary of the coupled system

Eq.	Roccon (2025)	Unknown	Implemented in
(26)	Eq. 1	ϕ	<code>phase_field.py::ac_rhs_2d/3d</code>
(27)	Eq. 2	$\dot{m}'''$	<code>phase_field.py::mdot_volumetric</code>
(28)	Eq. 3	(constraint)	<code>flow.py::ns_step_2d/3d</code> (Poisson RHS)
(31)	Eq. 4	$\mathbf{u}, p$	<code>flow.py::ns_step_2d/3d</code>
(32)	Eqs. 5–6	ρ, μ	<code>flow.py::rho,</code> <code>flow.py::mu</code>
(33)	Eqs. 7–8	$\mathbf{f}_\sigma$	<code>flow.py::surface_tension_2d/3d</code>
(34)	Eq. 9	T	<code>energy.py::energy_rhs_2d/3d</code>
(35)	Eq. 10	α	<code>energy.py::alpha_mix</code>
(36)	Eq. 11	S_t	<code>energy.py::energy_rhs_2d/3d</code>
(37)	Eq. 12	$\dot{m}$	<code>energy.py::mdot_from_heatflux_2d</code>

The coupling is genuinely two-way and tight: ϕ sets ρ, μ, α for the flow and energy equations; $\mathbf{u}$ advects ϕ and T ; T determines $\dot{m}$; and $\dot{m}$ sources both ϕ and the velocity divergence. The ordering in which these are advanced within a time step is therefore not arbitrary, and Section 9.7 examines it.

8 Numerical Methods

8.1 Spatial discretisation

All spatial derivatives are second-order central differences on a uniform Cartesian grid. For a field f with spacing h along a coordinate direction:

$$\left. \frac{\partial f}{\partial x} \right|_i = \frac{f_{i+1} - f_{i-1}}{2h} + \mathcal{O}(h^2), \quad (39)$$

$$\left. \frac{\partial^2 f}{\partial x^2} \right|_i = \frac{f_{i+1} - 2f_i + f_{i-1}}{h^2} + \mathcal{O}(h^2). \quad (40)$$

These are implemented once, generically over an array axis, in `src/operators.py`:

```
def _d1(f, h, axis):
    """First derivative along *axis* using 2nd-order central differences."""
    return (np.roll(f, -1, axis=axis) - np.roll(f, 1, axis=axis)) / (2 * h)

def _d2(f, h, axis):
    """Second derivative along *axis* using 2nd-order central differences."""
    return (np.roll(f, -1, axis=axis) - 2 * f + np.roll(f, 1, axis=axis)) / h**2
```

The use of `np.roll` deserves comment because it does two jobs simultaneously. It implements the stencil shift, and — because `np.roll` wraps — it *automatically imposes periodic boundary conditions* with no explicit halo handling. This is elegant and is exactly the right primitive for a triply-periodic domain. It is also the origin of a significant limitation: the code cannot represent any other boundary condition, which Sections 10.2 and 12.3 show is a real problem for the Stefan benchmark and a blocking problem for wall-bounded turbulence.

Central differences are chosen deliberately rather than as a default. They are **non-dissipative**: the modified-equation analysis of (39) produces a leading error term proportional to $\partial^3 f / \partial x^3$, which is dispersive, not diffusive. For a DNS this is essential — an upwind scheme’s numerical dissipation would contaminate the turbulent energy cascade at exactly the scales one is trying to resolve. Mirjalili et al. (2020) built their whole boundedness argument around central differencing for precisely this reason: they wanted a “non-dissipative discretization of advective terms” while still guaranteeing $\phi \in [0, 1]$.

The cost of that choice is that central differences provide *no damping at all* for the grid-scale ($2\Delta x$) mode: the modified wavenumber $\tilde{k} = \sin(kh)/h$ vanishes at $k = \pi/h$, so odd and even grid points decouple. If anything in the solution excites $2\Delta x$ oscillations, nothing in the scheme removes them. Section 11.2 shows this happening in the Stefan benchmark, and identifies the excitation mechanism.

The composite operators built on `_d1` and `_d2` are the obvious ones, with 2D and 3D variants throughout:

```
def grad_2d(f, dx, dy):      #  $\nabla f$ 
def div_2d(ux, uy, dx, dy): #  $\nabla \cdot u$ 
def laplacian_2d(f, dx, dy): #  $\nabla^2 f$ 
def grad_3d(f, dx, dy, dz)
```

```
def div_3d(ux, uy, uz, dx, dy, dz)
def laplacian_3d(f, dx, dy, dz)
```

Array layout is (N_y, N_x) in 2D and (N_z, N_y, N_x) in 3D, with x the last (fastest) axis — the C-contiguous layout NumPy prefers, so that x -direction stencils are cache-friendly.

One structural note for the eventual port. Roccon (2025) Section 2.5 uses a **staggered** grid: “velocity components are defined at cell faces while scalar fields (pressure, phase-field, and temperature) are defined at cell centers.” This repository uses a **collocated** grid, storing everything at the same points. Collocated grids are simpler but admit odd–even pressure–velocity decoupling (the classic checkerboard mode), which a staggered arrangement eliminates by construction. FLOW36 and MHIT36 are staggered. This difference is not currently causing observable trouble in the bubble benchmark, but it should be recorded as a known divergence from the reference method.

8.2 Why the pressure equation has constant coefficients

This is the key numerical property of the whole method, and it is worth deriving carefully because it is easy to take for granted.

The usual variable-density projection method. Write the momentum equation in non-conservative form, $\rho(\partial_t \mathbf{u} + \dots) = -\nabla p + \dots$, and apply Chorin’s projection (Chorin, 1968): advance to an intermediate velocity $\mathbf{u}^*$ ignoring pressure, then correct,

$$\frac{\mathbf{u}^{n+1} - \mathbf{u}^*}{\Delta t} = -\frac{1}{\rho} \nabla p.$$

Taking the divergence and enforcing $\nabla \cdot \mathbf{u}^{n+1} = 0$:

$$\nabla \cdot \left(\frac{1}{\rho} \nabla p \right) = \frac{\nabla \cdot \mathbf{u}^*}{\Delta t}. \quad (41)$$

This is a **variable-coefficient** elliptic equation. Its coefficient $1/\rho$ varies by a factor of 1000 across the interface. It does not diagonalise under any transform; it must be solved iteratively, and the coefficient jump makes it badly conditioned, so the iteration count is high. On a distributed GPU machine this is the bottleneck: every Krylov or multigrid iteration needs global reductions, and global reductions do not scale.

The one-fluid conservative-momentum formulation. Now take the *conservative* momentum variable $\mathbf{w} = \rho \mathbf{u}$ as primary, as in (31). The correction step becomes Eq. (R.17):

$$\frac{\mathbf{w}^{n+1} - \mathbf{w}^*}{\Delta t} = -\nabla p. \quad (42)$$

The density has disappeared from the correction, because p is being used to correct *momentum*, not velocity. Taking the divergence,

$$\frac{\nabla \cdot \mathbf{w}^{n+1} - \nabla \cdot \mathbf{w}^*}{\Delta t} = -\nabla^2 p,$$

and imposing that $\mathbf{w}^{n+1} = \rho^{n+1} \mathbf{u}^{n+1}$ satisfies mass conservation (28), i.e. $\nabla \cdot \mathbf{w}^{n+1} = \dot{m}'''^{n+1}(1 - \rho_v/\rho_l)$, gives Eq. (R.18):

$$\boxed{\nabla^2 p = \frac{\nabla \cdot \mathbf{w}^* - \dot{m}'''^{n+1}(1 - \rho_v/\rho_l)}{\Delta t}} \quad (43)$$

The operator on the left is the **plain, constant-coefficient Laplacian**. The entire density variation has been moved into the right-hand side (through $\mathbf{w}^*$) and into the final division (50). This is the property Roccon identifies as enabling “the use of fast, scalable and efficient FFT-based direct solvers”, and which he lists as one of the three reasons the method is suited to large-scale simulation.

Note also that the phase-change source enters (43) in exactly the right place: the second term in the numerator “represents the right-hand side of the mass conservation equation. This term is non-zero only in the interfacial region, where the occurrence of phase change induces a velocity jump.” The pressure field produced by the solve is precisely what generates the expansion flow (8) around a growing bubble.

The trade-off, stated honestly: the price of this formulation is that the correction (42) applies the same $\Delta t \nabla p$ to momentum on both sides of the interface, so the *velocity* correction is $\Delta t \nabla p / \rho$, which differs by a factor of 1000 across the interface. The resulting velocity field is less smooth near the interface than in a variable-coefficient formulation. In exchange, one gets an exact, non-iterative, perfectly-scalable pressure solve. For a code intended for thousands of GPUs, that is the right trade.

8.3 FFT solution of the Poisson equation

Because (43) has constant coefficients and periodic boundary conditions, the discrete Fourier basis diagonalises it exactly. This is not an approximation: applying the *discrete* Laplacian to a discrete Fourier mode gives that mode back, multiplied by an eigenvalue.

Take the 1D second-difference operator (40) acting on $f_j = e^{2\pi i k j / N}$:

$$\frac{f_{j+1} - 2f_j + f_{j-1}}{h^2} = \frac{e^{2\pi i k / N} - 2 + e^{-2\pi i k / N}}{h^2} f_j = \underbrace{\frac{2 \cos(2\pi k / N) - 2}{h^2}}_{\lambda_k} f_j. \quad (44)$$

So the eigenvalues of the discrete Laplacian are

$$\lambda_k = \frac{2 \cos(2\pi k / N) - 2}{h^2} = -\frac{4}{h^2} \sin^2\left(\frac{\pi k}{N}\right), \quad k = 0, 1, \dots, N-1. \quad (45)$$

Implemented verbatim in `src/pressure.py`:

```
def _eig_laplacian_1d(N, h):
    """Discrete eigenvalues of the 1-D 2nd-order Laplacian on N points."""
    k = np.fft.fftfreq(N) * N # integer wavenumbers: 0,1,...,N/2,-N/2+1,...,-1
    return (2 * np.cos(2 * np.pi * k / N) - 2) / h**2
```

An important detail: these are the eigenvalues of the **discrete** operator, not of the continuous one ($-k^2$). Using the discrete eigenvalues means the FFT solve inverts *exactly* the same operator that the rest of the code differentiates with. The discrete divergence of the corrected momentum field is then zero to machine precision — a consistency that would be lost if the continuous eigenvalues $-k^2$ were used instead.

Because the Laplacian is separable, the multi-dimensional eigenvalues are simply sums:

$$\Lambda_{k_x k_y k_z} = \lambda_{k_x}^{(x)} + \lambda_{k_y}^{(y)} + \lambda_{k_z}^{(z)}. \quad (46)$$

The solution algorithm is then three lines of linear algebra:

```
def solve_poisson_3d(rhs, dx, dy, dz):
    Nz, Ny, Nx = rhs.shape
    rhs_hat = np.fft.fftn(rhs)
    lam_x = _eig_laplacian_1d(Nx, dx)
    lam_y = _eig_laplacian_1d(Ny, dy)
    lam_z = _eig_laplacian_1d(Nz, dz)
    LAM = lam_z[:, None, None] + lam_y[None, :, None] + lam_x[None, None, :]
    LAM[0, 0, 0] = 1.0
    p_hat = rhs_hat / LAM
    p_hat[0, 0, 0] = 0.0
    return np.real(np.fft.ifftn(p_hat))
```

The two lines touching index `[0,0,0]` handle the null space. On a fully periodic domain the Laplacian is singular: constants are in its kernel, so $\Lambda_{000} = 0$ and pressure is determined only up to an additive constant. Setting `LAM[0,0,0] = 1.0` avoids division by zero, and setting `p_hat[0,0,0] = 0.0` afterwards selects the zero-mean solution from the family. This is the standard treatment and is correct; it also silently discards any inconsistency in the right-hand side (a non-zero mean of the RHS is unsolvable on a periodic domain and is here simply projected away), which is worth remembering as a place where a mass-conservation error could hide rather than announce itself.

The cost is $\mathcal{O}(N \log N)$ per solve with a small constant, compared with $\mathcal{O}(N)$ per multigrid V-cycle times an iteration count that grows with the density ratio — and, decisively, with no global reductions inside an iteration loop. The 3D version differs from the 2D one only in taking a 3D transform, which is exactly the “straightforward and directly GPU-portable” property claimed in the module docstring.

8.4 Time integration: the projection–correction scheme

The full time-advancement from n to $n + 1$ follows Roccon Section 2.5, Eqs. (R.14)–(R.20).

Step 1 — phase field, Eq. (R.14).

$$\phi^{n+1} = \phi^n + \Delta t A^n, \quad (47)$$

where A^n collects the advection, diffusion, sharpening and vaporisation terms of (26). Once ϕ^{n+1} is known, the density, viscosity and thermal diffusivity maps are re-evaluated.

Step 2 — energy, Eq. (R.15).

$$T^{n+1} = T^n + \Delta t B^n, \quad (48)$$

where B^n collects the advection and diffusion terms of (34). Saturation conditions are then imposed on the non-superheated phase — “equivalent to imposing a saturation Dirichlet boundary condition at the interface.”

Step 3 — momentum prediction, Eq. (R.16).

$$\frac{\mathbf{w}^* - \mathbf{w}^n}{\Delta t} = C^n, \quad (49)$$

with C^n the advection, viscous and surface-tension terms of (31).

Step 4 — pressure, Eq. (R.18). Solve (43) by FFT.

Step 5 — correction, Eqs. (R.19)–(R.20).

$$\mathbf{w}^{n+1} = \mathbf{w}^* - \Delta t \nabla p, \quad \mathbf{u}^{n+1} = \frac{\mathbf{w}^{n+1}}{\rho^{n+1}}. \quad (50)$$

Note the use of ρ^{n+1} , not ρ^n , in the final division: velocity at the new time level must be recovered using the density map that corresponds to the new phase field, or mass is not conserved consistently. `src/flow.py::ns_step_2d` takes both `phi_new` and `phi_old` as arguments precisely so it can do this correctly, and does.

8.5 Stability of the explicit scheme

Every equation is advanced by explicit Euler — first-order accurate in time, and conditionally stable. The binding restrictions are:

Convective CFL.

$$\Delta t < \frac{\Delta x}{|\mathbf{u}|_{\max}} \quad (51)$$

Viscous / diffusive. For a d -dimensional explicit diffusion,

$$\Delta t < \frac{\Delta x^2}{2d \nu_{\max}}, \quad \nu = \mu/\rho. \quad (52)$$

For the bubble benchmark parameters ($\mu_v = 10^{-3}$ Pa · s, $\rho_v = 1$ kg/m³) the kinematic viscosity of the *vapour* is $\nu_v = 10^{-3}$ m²/s — a thousand times that of the liquid. With $\Delta x = 1.5625 \times 10^{-4}$ m and $d = 2$ this gives $\Delta t < 6.1 \times 10^{-6}$ s. The benchmark uses $\Delta t = 5 \times 10^{-6}$ s, so this is the *binding* restriction, at 82% of the limit. This is worth knowing: the bubble benchmark is running close to the viscous stability boundary, and a modest grid refinement without a matching Δt reduction would destabilise it.

Phase-field sharpening. The sharpening flux acts like an advection at speed γ and the diffusive flux like a diffusion with coefficient $\gamma\epsilon$, giving $\Delta t < \Delta x/\gamma$ and $\Delta t < \Delta x^2/(2d\gamma\epsilon)$. With the (incorrectly small) γ used here these are never binding — which is itself diagnostic, since a correctly-sized $\gamma = |\mathbf{u}|_{\max}$ would make the sharpening term “the stiffest term in the equation”, exactly as Jain (2022) intends.

Surface tension. The Brackbill–Kothe–Zemach restriction (9). For water at the bubble-benchmark resolution, with $\rho_l + \rho_v \approx 1001$ kg/m³, $\Delta x = 1.5625 \times 10^{-4}$ m and $\sigma = 0.07$ N/m:

$$\Delta t_\sigma = \sqrt{\frac{1001 \times (1.5625 \times 10^{-4})^3}{2\pi \times 0.07}} \approx 9.3 \times 10^{-5} \text{ s}.$$

This is *not* restrictive at this resolution — it is about 15 times larger than the viscous limit. The much more severe estimate quoted in the repository README ($\Delta t \sim \sqrt{\rho_v \Delta x^3 / \sigma} \sim 10^{-9} \text{ s}$) uses the *vapour* density alone rather than the sum, which is the appropriate choice only in the opposite (light-fluid) limit; evaluated literally with $\rho_v = 1$ it gives $\Delta t_\sigma \approx 7.4 \times 10^{-6} \text{ s}$ — comparable to the viscous limit, but seven thousand times larger than the 10^{-9} s figure quoted. Section 12.2 revisits this, since the practical reason for setting $\sigma = 0$ turns out to be different from the one recorded in the README.

Overall. The prototype is stable for the bubble benchmark because the viscous limit is respected. It is first-order accurate in time throughout, which is adequate for a validation prototype but will need replacing by a higher-order Runge–Kutta scheme for turbulent DNS, where temporal accuracy affects the energy cascade directly.

9 Code Implementation

This section walks through the solver module by module, in dependency order, connecting every function to the equations of Sections 7 and 8. Where I have found the implementation to depart from Roccon (2025) or to contain an outright error, I say so at the point where the code appears, and carry the item forward into the limitations of Section 12. Several of these were found while writing this document and running the studies of Section 11; recording them precisely is the main reason the prototype exists.

9.1 `src/params.py` — the parameter container

Every physical and numerical parameter lives in a single `SimParams` dataclass, which is threaded through every function as the argument `p`. This keeps the call signatures short and makes it impossible for two modules to disagree about, say, the interface thickness.

```
@dataclass
class SimParams:
    ndim: int = 2
    Nx: int = 64
    ...
    rho_l: float = 1000.0 # liquid density [kg/m³]
    rho_v: float = 1.0 # vapour density [kg/m³]
    ...
    eps: float = None # interface width [m] (default: 1.5 × dx)
    gamma: float = None # Allen-Cahn mobility (default: eps)
    mode: str = 'prescribed'
    mdot_surf: float = 0.1
```

The physical defaults are water at saturation and standard pressure, matching the bubble-growth benchmark of Roccon (2025) Section 3.4: `rho_l=1000.0`, `rho_v=1.0` (density ratio 10^{-3}), `sigma=0.07`, `h_lv=2.25e6`, `T_sat=373.15`. The thermal conductivities `k_l=0.677` and `k_v=0.024` match the water properties Roccon uses in his adsorption benchmark.

Grid spacings are derived properties rather than stored fields, which guarantees they can never fall out of sync with the domain size and resolution:

```
@property
def dx(self) -> float:
    return self.Lx / self.Nx
```

The `mode` field selects the vaporisation-rate closure: `'prescribed'` fixes $\dot{m}$ directly (the bubble benchmark), `'heat_flux'` computes it from (37) (the Stefan problem and, eventually, turbulent boiling).

9.1.1 Defect 1: `gamma = eps` is dimensionally inconsistent

```
def __post_init__(self):
    if self.eps is None:
        self.eps = 1.5 * self.dx
    if self.gamma is None:
```



```
self.gamma = self.eps
```

The first default is correct and matches Roccon (2025), who sets $\varepsilon = 1.5\Delta x$ for the Stefan and adsorption benchmarks. The second is not.

As established in Section 4.5, γ is a **velocity-scale** parameter with units of m/s; Jain (2022) names it exactly that, and dimensional analysis of (23) forces $[\gamma] = L/T$. Assigning it the value of ε , a length, is a units error that happens to produce a finite number.

The numerical consequence is that γ is far too small. For the bubble benchmark, $\varepsilon = 1.5 \times 1.5625 \times 10^{-4} = 2.34 \times 10^{-4}$, so $\gamma = 2.34 \times 10^{-4}$ m/s, while the interface itself moves at $\dot{m}/\rho_v = 0.1$ m/s. The boundedness criterion (24) with $\epsilon^* = 1.5$ requires $\Gamma^* \geq 1/(2 \times 1.5 - 1) = 0.5$, i.e. $\gamma \geq 0.5 |\mathbf{u}|_{\max} \approx 0.05$ m/s. The implemented γ is roughly **200 times smaller than required**.

The practical effect is that the sharpening flux is far too weak to maintain the equilibrium tanh profile against advection: the interface is free to smear. Boundedness of ϕ is nonetheless maintained in practice, but only because `src/solver.py` applies a hard `np.clip(phi + p.dt * A, 0.0, 1.0)` after every step — that is, the analytical guarantee of Mirjalili et al. (2020) has been replaced by a numerical band-aid. The clip prevents negative densities, which is why the bubble benchmark runs at all, but it is not the same thing as a bounded scheme, and it introduces a small non-conservative mass error each time it activates.

The fix is one line, and should be applied before the Fortran port:

```
# gamma is a velocity scale (Jain 2022: Gamma >= |u|_max)
if self.gamma is None:
    self.gamma = max(self.mdot_surf / self.rho_v, self.dx / self.dt * 1e-3)
```

or, more robustly, recomputing $\gamma = |\mathbf{u}|_{\max}$ each step inside the solver loop, which is what MHIT36 does.

9.2 `src/operators.py` — spatial discretisation

Discussed in Section 8.1. The module is 65 lines and contains no state. Two private helpers `_d1` and `_d2` implement (39) and (40) generically over a NumPy axis; everything else is a thin composition. The 2D and 3D variants are separate functions rather than one dimension-agnostic implementation, which costs a little duplication but makes the array-axis conventions explicit — a worthwhile trade, since axis confusion is the single most common source of silent errors in this kind of code.

The axis convention is fixed and documented at module level: `x` is the last axis, `y` the second-to-last, `z` the third-to-last, so arrays are `(Ny, Nx)` in 2D and `(Nz, Ny, Nx)` in 3D. Note that `examples/bubble_2d.py` builds its initial condition with `np.meshgrid(x, y)` (default 'xy' indexing, giving shape `(Ny, Nx)`), which is consistent; the 3D snippet in the README uses `np.meshgrid(..., indexing='ij')`, giving shape `(Nx, Ny, Nz)`, which is **not** consistent with the `(Nz, Ny, Nx)` layout the 3D operators assume. For an isotropic initial condition on a cubic domain this transposition is invisible, but it will produce silently wrong results on a non-cubic grid, and should be corrected in the README before anyone follows it.

9.3 src/phase_field.py — conservative Allen–Cahn

This module implements (26) and (27). The volumetric source conversion is a direct transcription of Eq. (R.2):

```
def mdot_volumetric(phi, mdot_surf, eps):
    """Convert surface vaporisation rate to volumetric via  $|\nabla\phi| \approx \phi(1-\phi)/\epsilon$  (Eq. 2)."""
    return mdot_surf * phi * (1 - phi) / eps
```

The right-hand side assembly is worth reading closely, because the algebra of the flux is compressed:

```
def ac_rhs_2d(phi, ux, uy, mdot_vol, p):
    dphix = grad_x_2d(phi, dx)
    dphiy = grad_y_2d(phi, dy)
    mag = np.sqrt(dphix**2 + dphiy**2 + 1e-14) #  $|\nabla\phi|$ 

    # Total Allen-Cahn flux  $J = \gamma(\epsilon \nabla\phi - \phi(1-\phi) \hat{n})$ 
    coeff = phi * (1 - phi) / mag
    Jx = gam * (eps * dphix - coeff * dphix)
    Jy = gam * (eps * dphiy - coeff * dphiy)

    div_J = grad_x_2d(Jx, dx) + grad_y_2d(Jy, dy)
    adv = -(grad_x_2d(ux * phi, dx) + grad_y_2d(uy * phi, dy))
    source = mdot_vol / p.rho_v
    return adv + div_J + source
```

The identity being used is that the normal is $\hat{n}_x = \partial_x \phi / |\nabla \phi|$, so the sharpening flux component is $-\gamma \phi(1-\phi)\hat{n}_x = -\gamma [\phi(1-\phi)/|\nabla \phi|] \partial_x \phi = -\gamma \text{coeff} \cdot \text{dphix}$. Hence $J_x = \gamma \partial_x \phi (\epsilon - \text{coeff})$, which is what the code computes. This is correct, and the factored form is also numerically better behaved than computing $\hat{n}$ explicitly, since the potentially-singular division by **mag** appears once rather than twice.

The **1e-14** regularisation inside the square root prevents division by zero in the bulk phases, where $\nabla \phi = \mathbf{0}$ exactly. Note that in the bulk this makes **coeff** $\rightarrow \phi(1-\phi)/10^{-7} \rightarrow 0$ as well, since $\phi(1-\phi) = 0$ there, so the flux correctly vanishes. The regularisation is doing real work only in the thin region where ϕ is intermediate but $\nabla \phi$ is momentarily small — for example at a saddle between two merging bubbles.

The advection term is written in **conservative** form, $-\nabla \cdot (\mathbf{u}\phi)$ rather than $-\mathbf{u} \cdot \nabla \phi$, which matters: it is what makes $d/dt \int \phi dV$ telescope to zero on a periodic grid. Combined with **div_J** also being a discrete divergence, the entire right-hand side except the source is discretely conservative, which is the defining property of the CAC formulation.

ac_rhs_3d is the exact analogue with a third component; no structural differences.

9.4 src/pressure.py — the FFT Poisson solver

Discussed in detail in Section 8.3. Seventy-three lines, no dependencies beyond NumPy, and the most directly portable module in the repository — **np.fft.fftn** maps onto **cuFFT** and the eigenvalue construction is a trivially parallel elementwise operation. This module is the concrete embodiment of the argument in Section 8.2 that motivates the entire method.

The only judgement call in it is the null-space handling at $[0,0]/[0,0,0]$, discussed in Section 8.3. It is standard and correct for a periodic domain, but it does mean that a right-hand side with a non-zero mean — which would signal a violation of global mass conservation — is silently projected away rather than raising. Adding an assertion on `abs(rhs.mean())` would be a cheap and worthwhile diagnostic to carry into the Fortran version.

9.5 `src/flow.py` — Navier–Stokes projection–correction

This is the largest module (198 lines) and implements (31) through (50). The mixture properties are one-liners transcribing Eqs. (R.5)–(R.6):

```
def rho(phi, p):
    """ $\rho(\phi) = \rho_v \phi + \rho_l(1-\phi)$  (Eq. 5)."""
    return p.rho_v * phi + p.rho_l * (1 - phi)

def mu(phi, p):
    """ $\mu(\phi) = \mu_v \phi + \mu_l(1-\phi)$  (Eq. 6)."""
    return p.mu_v * phi + p.mu_l * (1 - phi)
```

Curvature and the CSF force implement Eqs. (R.7)–(R.8):

```
def _curvature_2d(phi, dx, dy):
    """ $\kappa = \nabla \cdot (\nabla \phi / |\nabla \phi|)$ ."""
    dphix = grad_x_2d(phi, dx); dphiy = grad_y_2d(phi, dy)
    mag = np.sqrt(dphix**2 + dphiy**2 + 1e-14)
    return grad_x_2d(dphix / mag, dx) + grad_y_2d(dphiy / mag, dy)

def surface_tension_2d(phi, p):
    """ $f\sigma = 6\sigma \kappa \phi(1-\phi) \nabla \phi$  (Eq. 7) — returns (fx, fy)."""
    kappa = _curvature_2d(phi, p.dx, p.dy)
    c = 6 * p.sigma * kappa * phi * (1 - phi)
    return c * grad_x_2d(phi, p.dx), c * grad_y_2d(phi, p.dy)
```

The main time-step function follows Eqs. (R.16)–(R.20) in order, and its signature encodes the important detail that both phase-field levels are needed:

```
def ns_step_2d(phi_new, phi_old, ux, uy, mdot_vol, p):
    rho_n = rho(phi_old, p)
    mu_n = mu(phi_old, p)
    rho_np = rho(phi_new, p)    #  $\rho^{n+1}$ 
    wx = rho_n * ux; wy = rho_n * uy    #  $w = \rho u$ 
    ...
    wx_star = wx + dt * Cx    # Eq. 16
    mass_src = mdot_vol * (1.0 - p.rho_v / p.rho_l)    # RHS of Eq. 3
    div_w_star = div_2d(wx_star, wy_star, dx, dy)
    rhs_p = (div_w_star - mass_src) / dt
    pres = solve_poisson_2d(rhs_p, dx, dy)    # Eq. 18
    wx_new = wx_star - dt * grad_x_2d(pres, dx)    # Eq. 19
    ux_new = wx_new / rho_np    # Eq. 20
    return ux_new, uy_new, pres
```

The pressure right-hand side is exactly (43), including the phase-change term, and the final division uses ρ^{n+1} as Eq. (R.20) requires. This is the core of the method and it is implemented correctly.

9.5.1 Defect 2: momentum advection uses the non-conservative form

```
# --- Advection:  $-\nabla \cdot (\rho \mathbf{u} \mathbf{u}) = -\rho(\mathbf{u} \cdot \nabla) \mathbf{u}$  ---
adv_x = -rho_n * (ux * grad_x_2d(ux, dx) + uy * grad_y_2d(ux, dy))
```

The comment asserts an identity that does not hold here. Expanding the conservative form,

$$\nabla \cdot (\rho \mathbf{u} \mathbf{u}) = \rho(\mathbf{u} \cdot \nabla) \mathbf{u} + \mathbf{u} \nabla \cdot (\rho \mathbf{u}), \quad (53)$$

the two forms agree only when $\nabla \cdot (\rho \mathbf{u}) = 0$. But that is precisely what (28) says is *not* true during phase change: $\nabla \cdot (\rho \mathbf{u}) = \dot{m}'''(1 - \rho_v/\rho_l)$. The implementation therefore omits the term

$$\mathbf{u} \dot{m}''' \left(1 - \frac{\rho_v}{\rho_l} \right), \quad (54)$$

which is the momentum carried by the mass crossing the interface. It is non-zero exactly where phase change occurs, and it scales with the interface velocity. In the bubble benchmark it is small because the flow is nearly radially symmetric and the omitted term is largely balanced by the pressure field; it is nonetheless a genuine inconsistency with Eq. (R.4) and contributes to the growth-rate deficit measured in Section 11.1. The fix is to compute the advection as a true divergence of the momentum flux tensor, $-\left[\partial_x(\rho u_x u_j) + \partial_y(\rho u_y u_j)\right]$.

9.5.2 Defect 3: the 3D viscous term drops the variable-viscosity correction

The 2D viscous term retains a first-order correction for spatially varying viscosity:

```
visc_x = (mu_n * laplacian_2d(ux, dx, dy)
          + grad_x_2d(mu_n, dx) * grad_x_2d(ux, dx)
          + grad_y_2d(mu_n, dy) * grad_y_2d(ux, dy))
```

whereas the 3D version does not:

```
visc_x = mu_n * laplacian_3d(ux, dx, dy, dz)
```

Neither is the full $\nabla \cdot [\mu(\nabla \mathbf{u} + \nabla \mathbf{u}^T)]$ — both drop the transpose contribution $\nabla \cdot (\mu \nabla \mathbf{u}^T)$, which for a divergence-free flow reduces to $\nabla \mu \cdot (\nabla \mathbf{u})^T$ and is not negligible across an interface where μ jumps. But the 3D version is strictly worse than the 2D one, and the inconsistency between them means that a 2D and a 3D run of the same physical problem would not agree even in the limit of an infinitely thin third dimension. Since both benchmarks currently use matched viscosities ($\mu_l = \mu_v$), $\nabla \mu = \mathbf{0}$ and the defect is dormant — but it will activate the moment realistic water/steam viscosities are used, which is the point of the exercise. This must be fixed before any 3D validation is attempted.

9.6 src/energy.py — energy equation and vaporisation rate

The mixture diffusivity is Eq. (R.10):

```
def alpha_mix(phi, p):
    alpha_v = p.k_v / (p.rho_v * p.Cp_v)
    alpha_l = p.k_l / (p.rho_l * p.Cp_l)
    return alpha_v * phi + alpha_l * (1 - phi)
```

and the energy right-hand side assembles (34):

```
def energy_rhs_2d(T, phi, ux, uy, ac_rhs, p):
    adv = -(ux * grad_x_2d(T, dx) + uy * grad_y_2d(T, dy))
    dTx = grad_x_2d(T, dx); dTy = grad_y_2d(T, dy)
    diff = grad_x_2d(alpha * dTx, dx) + grad_y_2d(alpha * dTy, dy)
    St = -(p.h_lv / Cp) * ac_rhs
    return adv + diff + St
```

The diffusion term is correctly written as $\nabla \cdot (\alpha \nabla T)$ — a nested divergence of a flux, not $\alpha \nabla^2 T$ — which matters because α varies by a factor of 40 across the interface for water. Note however that the advection term is the *non-conservative* $-\mathbf{u} \cdot \nabla T$, whereas Eq. (R.9) specifies the conservative $-\nabla \cdot (\mathbf{u}T)$; these differ by $T \nabla \cdot \mathbf{u}$, which is again non-zero in the interfacial region during phase change.

9.6.1 Defect 4: the latent-heat source does not match Eq. (R.11)

The docstring states that the `ac_rhs` argument is “ $A = \partial\phi/\partial t + \nabla \cdot (\mathbf{u}\phi)$ (net, no source)”. What `src/solver.py` actually passes is

```
A_no_src = ac_rhs_2d(phi, ux, uy, np.zeros_like(mdot_vol), p)
```

i.e. the Allen–Cahn right-hand side with the source zeroed, which by construction equals $-\nabla \cdot (\mathbf{u}\phi) + \nabla \cdot \mathbf{J}$. But from (26), the quantity Eq. (R.11) actually requires is

$$\frac{\partial\phi}{\partial t} + \nabla \cdot (\mathbf{u}\phi) = \nabla \cdot \mathbf{J} + \frac{\dot{m}'''}{\rho_v}. \quad (55)$$

The code computes $\nabla \cdot \mathbf{J} - \nabla \cdot (\mathbf{u}\phi)$ instead. The two differ in both the sign of the advective contribution and by the omission of the vaporisation source — which is the dominant term, and the only one with any latent-heat meaning. The implemented S_t is therefore not Eq. (R.11).

The practical severity is limited, because Roccon (2025) argues that under the saturation assumption S_t “can be neglected” entirely, and the solver’s saturation overwrite is what actually enforces the interfacial temperature condition. But an incorrect non-zero S_t is strictly worse than a correctly-omitted one: it injects a spurious temperature source of magnitude $\sim (h_{lv}/C_p)\nabla \cdot \mathbf{J}$, and with $h_{lv}/C_p \approx 534$ K for water, even a small $\nabla \cdot \mathbf{J}$ produces a significant spurious heating or cooling in the interfacial layer. Either implement (36) faithfully or omit S_t explicitly with a comment; the present middle ground is the worst option.

9.6.2 Defect 5: `mdot_from_heatflux_2d` is wrong, and vanishes identically when $k_v = k_l$

This is the most consequential error found, and it is worth stating carefully.

```
def mdot_from_heatflux_2d(T, phi, p):
    dTx = grad_x_2d(T, dx); dTy = grad_y_2d(T, dy)
```

```

dphix = grad_x_2d(phi, dx); dphiy = grad_y_2d(phi, dy)
mag    = np.sqrt(dphix**2 + dphiy**2 + 1e-14)
nx, ny = dphix / mag, dphiy / mag
grad_T_n = dTx * nx + dTy * ny          #  $(\nabla T) \cdot \hat{n}$ 
mdot_surf = (p.k_v - p.k_l) * grad_T_n / p.h_lv
mdot_vol  = mdot_surf * phi * (1 - phi) / p.eps
return mdot_vol

```

Equation (R.12) is $\dot{m} = (k_v \nabla T_v - k_l \nabla T_l) \cdot \mathbf{n} / h_{lv}$, with ∇T_v and ∇T_l **two distinct one-sided gradients** evaluated in the vapour and in the liquid respectively. The implementation collapses them into a single central-difference gradient ∇T and factors it out, giving $(k_v - k_l) \nabla T \cdot \mathbf{n} / h_{lv}$. That factorisation is valid only if $\nabla T_v = \nabla T_l$, which is exactly the condition that *never* holds at a phase-change interface — the whole physical content of the Rankine–Hugoniot balance is that the two gradients differ.

The consequence is stark for the very benchmark the function is meant to serve. The Stefan setup uses matched conductivities, $k_l=0.005$, $k_v=0.005$, so $(k_v - k_l) = 0$ **exactly**, and the function returns identically zero: no phase change whatsoever. Any call to `run_2d(..., mode='heat_flux')` with matched conductivities produces a frozen interface.

The reason this has not been noticed is that `examples/stefan_1d.py` never calls it. The example inlines its own, physically correct, single-sided form:

```

# In the Stefan problem the liquid is at T_sat ( $\nabla T_{\text{liquid}} = 0$ ), so only
# the vapour-side heat flux drives vaporisation:  $\dot{m} = k_v (\nabla T \cdot \hat{n}) / h_{lv}$ 
mdot_surf_local = p.k_v * dT_dx * nx / p.h_lv

```

That inline version is correct *for this particular benchmark*, where the liquid is held at T_{sat} so $\nabla T_l = 0$ and Eq. (R.12) genuinely reduces to $k_v \nabla T_v \cdot \mathbf{n} / h_{lv}$. So the benchmark exercises a correct closure while the library function it is supposed to be validating is never touched. Additionally, with the *default* water properties ($k_v = 0.024$, $k_l = 0.677$), the library function returns $(k_v - k_l) < 0$, so it would predict condensation where vaporisation should occur — a sign error on top of the structural one.

The correct fix is the probe method of Section 7.5, which is the Year-1 task already identified in the repository README. In the interim, the function should at minimum be rewritten to evaluate one-sided gradients using ϕ -weighted masks, and should raise rather than silently return zero when $k_v = k_l$.

9.7 src/solver.py — the coupled time loop

`run_2d` implements the operator ordering of Roccon (2025) Section 2.5. The ordering is the substantive content of this module, and it is not arbitrary:

```

for step in range(n_steps):
    # — 1. Vaporisation rate —
    if p.mode == 'prescribed':
        mdot_vol = mdot_volumetric(phi, p.mdot_surf, p.eps)
    else:
        mdot_vol = mdot_from_heatflux_2d(T, phi, p)

```

```

# — 2. Allen-Cahn (Eq. 14) —
A = ac_rhs_2d(phi, ux, uy, mdot_vol, p)
phi_new = np.clip(phi + p.dt * A, 0.0, 1.0)

# — 3. Energy equation (Eq. 15) —
...
T_new = np.where(phi_new < 0.5, p.T_sat, T_new)

# — 4. Navier-Stokes (Eqs. 16–20) —
ux_new, uy_new, pres = ns_step_2d(phi_new, phi, ux, uy, mdot_vol, p)

```

Why this order matters. The phase field is advanced *first* because everything downstream needs ϕ^{n+1} : the density map ρ^{n+1} used in the final velocity recovery (50), the saturation mask that decides which cells are held at T_{sat} , and the property maps for the energy equation. Roccon states this explicitly: “Once obtained ϕ^{n+1} , the density, viscosity and thermal diffusivity maps are evaluated.”

Energy is advanced *second* because the vaporisation rate for the *next* step depends on T^{n+1} , and because the saturation overwrite must be applied against the new phase field ϕ^{n+1} — applying it against ϕ^n would leave a one-cell band of stale superheated liquid behind a moving interface.

Navier–Stokes is advanced *last* because its pressure equation (43) requires $\dot{m}'''^{n+1}$ on the right-hand side, and because the velocity recovery needs ρ^{n+1} . Reversing the order would make the mass source in the Poisson equation inconsistent with the phase field it is supposed to correspond to, and the resulting velocity field would not satisfy (28).

The one deviation from the paper is that `mdot_vol` is computed from ϕ^n and T^n at the top of the step and then reused in the pressure equation, where Eq. (R.18) calls for $\dot{m}'''^{n+1}$. This is a first-order-in-time lag, consistent with the overall first-order scheme, and is not an error at this accuracy — but it should be noted when moving to a higher-order Runge–Kutta scheme, where such lags become the accuracy-limiting term.

9.7.1 Defect 6: the velocity clamp is N_x times looser than documented

```

# Clamp velocity to prevent blow-up in explicit scheme
u_max = np.sqrt(np.max(ux_new**2 + uy_new**2) + 1e-30)
u_lim = p.Lx / p.dt # ~ 1 grid cell per step

```

The comment says “1 grid cell per step”, which would be `p.dx / p.dt`. The code uses `p.Lx / p.dt` — one *domain length* per step, a limit $N_x = 64$ times larger. As written the clamp is essentially inert: it will only trigger after the solution has already diverged catastrophically. This is not currently causing a problem (the bubble benchmark is stable on its own merits, as Section 8.5 shows), but it means the safety net that the comment promises does not exist. Either fix the constant or remove the clamp; a silently non-functional guard is worse than none, because it invites false confidence.

Note also that any active clamp of this kind is non-conservative — it rescales momentum without a corresponding pressure adjustment, so the velocity field it produces no longer satisfies (28). A production version should reduce Δt rather than clamp $\mathbf{u}$.

9.7.2 Defect 7: unreachable code in `run_3d`

```
if p.mode == 'prescribed':
    mdot_vol = mdot_volumetric(phi, p.mdot_surf, p.eps)
else:
    raise NotImplementedError("heat_flux mode for 3-D: implement probe method")
...
if p.mode == 'heat_flux':          # ← unreachable
    A_ns = ac_rhs_3d(...)
    B     = energy_rhs_3d(...)
```

The `raise` above guarantees that `p.mode == 'heat_flux'` can never reach the energy block, so the entire 3D energy path is dead code and has never executed. This is honest about the current state — 3D boiling genuinely is not implemented — but it means the 3D energy equation is completely untested. Any claim that `run_3d` is “implemented, needs testing” should be read as applying only to the ‘prescribed’ path.

9.8 `src/diagnostics.py` — post-processing

Five small functions, of which two are used by the benchmarks.

```
def bubble_radius_2d(phi, dx, dy):
    """Estimate bubble radius from the vapour-phase area  $A = \pi R^2$ ."""
    area = np.sum(phi) * dx * dy
    return np.sqrt(area / np.pi)
```

This is an **integral** measure: it computes the total vapour area $A = \int \phi dA$ and inverts $A = \pi R^2$. The choice is deliberate and good — an integral measure is far less noisy than locating the $\phi = 0.5$ contour, and because the conservative Allen–Cahn equation conserves $\int \phi dV$ exactly, this diagnostic directly measures the quantity the scheme is designed to preserve.

It does, however, carry a systematic bias at finite ε in curved geometry, which Section 11.1 quantifies and which turns out to explain most of the apparent error in the bubble benchmark. The 3D analogue inverts $V = \frac{4}{3}\pi R^3$ identically.

```
def interface_position_1d(phi_1d, x):
    f = phi_1d - 0.5
    idx = np.where(np.diff(np.sign(f)))[0]
    if len(idx) == 0:
        return None
    i = idx[0]
    frac = -f[i] / (f[i + 1] - f[i])
    return x[i] + frac * (x[i + 1] - x[i])
```

Linear interpolation to the $\phi = 0.5$ crossing, returning the **first** crossing found. On a clean monotone profile this is exactly right. Once the field develops spurious oscillations, however, `idx[0]` picks whichever crossing happens to lie leftmost, which may have nothing to do with the physical interface. Section 11.2 shows this failure mode occurring in the Stefan benchmark, where the reported interface position freezes at the location of the first spurious crossing while the real vapour region continues to grow. A more robust diagnostic would use the integral measure $\int \phi dx$, which

for a 1D problem is exactly δ when the profile is a single clean front, and which degrades gracefully rather than catastrophically.

The remaining functions — `nusselt_number_2d`, `lp_norm`, `kinetic_energy_2d` — are utilities prepared for later parameter studies and are not exercised by the current benchmarks. `nusselt_number_2d` assumes a wall at row index 0 with a one-sided gradient, which is inconsistent with the periodic boundary conditions the rest of the code imposes; it is a placeholder awaiting the wall boundary conditions of Section 13.3.

10 Validation Experiments

Two benchmarks from Roccon (2025) are implemented. They are complementary: the bubble tests the *hydrodynamic* coupling (mass source, expansion flow, pressure) with the vaporisation rate prescribed, while the Stefan problem tests the *thermal* coupling (conduction, latent heat, heat-flux closure) with the flow suppressed. Together they exercise every term in the system except surface tension.

10.1 Benchmark 1: 2D vapour bubble growth at constant vaporisation rate

Setup. A circular vapour bubble of initial radius $R_0 = 1$ mm sits at the centre of a 10×10 mm square domain, surrounded by liquid. A spatially uniform, temporally constant vaporisation rate per unit surface $\dot{m} = 0.1 \text{ kg m}^{-2} \text{ s}^{-1}$ is imposed. Densities are $\rho_l = 1000$, $\rho_v = 1 \text{ kg/m}^3$ (ratio 10^{-3} , water at saturation and standard pressure); viscosities are matched at $10^{-3} \text{ Pa} \cdot \text{s}$. This replicates Roccon (2025) Section 3.4, which in turn replicates the benchmark of Tanguy et al. (2014).

Analytical solution. Because $\dot{m}$ is uniform over the interface and constant in time, the derivation is elementary. The mass of vapour created per unit time is $\dot{M} = \dot{m} S(t)$ where $S(t)$ is the interfacial length (in 2D, per unit depth, $S = 2\pi R$). That mass occupies volume $\dot{M}/\rho_v$, so the vapour area grows as

$$\frac{dA}{dt} = \frac{\dot{m} 2\pi R}{\rho_v}, \quad A = \pi R^2 \implies 2\pi R \frac{dR}{dt} = \frac{2\pi R \dot{m}}{\rho_v}, \quad (56)$$

whence $dR/dt = \dot{m}/\rho_v$, a constant, and

$$\boxed{R(t) = R_0 + \frac{\dot{m}}{\rho_v} t} \quad (57)$$

This is Eq. (R.33). The interface velocity is $\dot{m}/\rho_v = 0.1 \text{ m/s}$, so the bubble doubles its radius in exactly 10 ms. Note that the result is independent of surface tension, viscosity, and liquid density — which is what makes it a clean test of the mass source and the expansion flow in isolation, and what justifies running it with $\sigma = 0$.

The expansion flow it must generate is also known analytically. Mass conservation in the liquid, with the interface receding at dR/dt , gives a radial velocity

$$u_r(r) = \frac{\dot{m}}{\rho_v} \left(1 - \frac{\rho_v}{\rho_l} \right) \frac{R}{r}, \quad r > R, \quad (58)$$

i.e. $\approx 0.1 \text{ m/s}$ at the interface decaying as $1/r$ — a direct consequence of (8) and hence of the source term in (43). If the pressure solver were wrong, the bubble would not grow at the right rate regardless of what the phase-field equation did.

Implementation. `examples/bubble_2d.py`, at 64×64 resolution with $\Delta t = 5 \times 10^{-6} \text{ s}$, run to $t = 8 \text{ ms}$. Surface tension is set to zero; the in-code comment gives the reason and it is worth quoting since Section 12.2 revisits it:

```

sigma=0.0,    # sigma=0 here: analytical R(t) is independent of surface tension.
              # Non-zero sigma with explicit CSF requires dt ~ sqrt(rho_v*dx^3/sigma)
              # ~ 1e-9 s - implicit surface tension is PhD Year-2 work.

```

The measured radius comes from `bubble_radius_2d`, compared against (57) in the callback:

```

def callback(step, t, state):
    R = bubble_radius_2d(state['phi'], p.dx, p.dy)
    Ra = R0 + (p.mdot_surf / p.rho_v) * t

```

Deviations from the paper. Three, all deliberate or forced: Roccon uses $\varepsilon = \Delta x$ where this repository defaults to $1.5\Delta x$; Roccon imposes outflow boundary conditions ($p = 0$, $\partial \mathbf{u} / \partial n = 0$) on all sides where this code is periodic; and Roccon uses $\sigma = 0.07$ N/m where this code uses zero. The periodic boundary condition is the significant one — a growing bubble in a periodic box pushes fluid out of one face and back in through the opposite one, whereas outflow conditions let it escape. At $R \leq 1.75$ mm in a 10 mm box the bubble occupies under 10% of the domain, so confinement is mild, but it is not zero and it acts to retard growth.

10.2 Benchmark 2: 1D Stefan problem

Setup. A thin vapour layer sits against a wall held at $T_w > T_{\text{sat}}$; the rest of the domain is liquid at T_{sat} . The superheated vapour conducts heat to the interface, vaporising liquid, and the interface advances away from the wall. The vapour is motionless; the expansion drives liquid toward the outlet.

Analytical solution. This is a classical similarity solution and is worth deriving because both the interface law and the transcendental equation appear in the code.

In the vapour layer $0 < x < \delta(t)$, temperature obeys pure conduction $\partial_t T = \alpha_v \partial_{xx} T$ with $T(0, t) = T_w$ and $T(\delta, t) = T_{\text{sat}}$. Seeking a similarity solution in $\eta = x / (2\sqrt{\alpha_v t})$ gives $T = T_w + C \text{erf}(\eta)$. Imposing the interface condition requires $\delta(t) \propto \sqrt{\alpha_v t}$; writing

$$\boxed{\delta(t) = 2\xi\sqrt{\alpha_v t}}, \quad \alpha_v = \frac{k_v}{\rho_v C_{p,v}}, \quad (59)$$

(Eq. R.24) and fixing C from $T(\delta) = T_{\text{sat}}$ gives the vapour temperature profile (Eq. R.27):

$$T(x, t) = T_w - \frac{T_w - T_{\text{sat}}}{\text{erf}(\xi)} \text{erf}\left(\frac{x}{2\sqrt{\alpha_v t}}\right). \quad (60)$$

The constant ξ is fixed by the energy jump condition (6). Since the liquid is at T_{sat} uniformly, $\nabla T_l = 0$, and the balance reduces to $\rho_v h_{lv} d\delta/dt = k_v \partial_x T|_{x=\delta-}$. Differentiating (60) and (59) and substituting yields the transcendental equation (Eqs. R.25–R.26):

$$\boxed{\xi \exp(\xi^2) \text{erf}(\xi) = \frac{\text{St}}{\sqrt{\pi}}}, \quad \text{St} = \frac{C_{p,v}(T_w - T_{\text{sat}})}{h_{lv}}, \quad (61)$$

where St is the **Stefan number**, the ratio of sensible to latent heat. The liquid velocity follows from mass conservation (Eq. R.28): $u_l = (1 - \rho_v/\rho_l) \xi \sqrt{\alpha_v/t}$.

Parameters. `examples/stefan_1d.py` reproduces Roccon’s Section 3.2 setup precisely: $L_x = 0.2$ m, $N_x = 200$, $\rho_l = \rho_v = 1$ kg/m³, $\mu = 0.01$ Pa · s, $k_v = k_l = 0.005$ W/(m · K), $C_{p,v} = 200$ J/(kg · K), $h_{lv} = 10^4$ J/kg, $\Delta T = 10$ K. These give $St = 0.2$ and, from (61), $\xi = 0.306424$ — matching the value $\xi = 0.3064$ reported in the paper exactly. The thermal diffusivity is $\alpha_v = 2.5 \times 10^{-5}$ m²/s. Matched densities mean no expansion flow, so the Navier–Stokes solve can be skipped entirely and the example integrates only the phase-field and energy equations.

The transcendental equation is solved numerically with Brent’s method:

```
xi = brentq(lambda xi: xi * np.exp(xi**2) * erf(xi) - St_num / np.sqrt(np.pi),
            1e-8, 10.0)
```

A parameter discrepancy. The example starts the integration at $t_0 = 24.7$ s, with the comment “(matching paper Section 3.2)”. That start time is in fact taken from Roccon’s **Section 3.3**, the *adsorption* problem, where it corresponds to an interface at $x = 0.1$ m in the Irfan and Muradoglu (2017) setup. The Stefan problem in Roccon’s Section 3.2 starts at $t \approx 0$ with the interface initialised at the wall (“for computational reasons, the leftmost cells are initialized as vapor”). At $t_0 = 24.7$ s the Stefan similarity solution gives $\delta = 15.2$ mm, not 100 mm. Starting from a self-consistent similarity state at $t_0 = 24.7$ s is a legitimate way to avoid the $t \rightarrow 0$ singularity and does not invalidate the test, but the code comment attributes the choice to the wrong benchmark and should be corrected.

Measurement. `interface_position_1d` locates the $\phi = 0.5$ crossing and compares it against (59). As noted in Section 9.8, this diagnostic is fragile once the solution loses monotonicity, and Section 11.2 shows exactly that happening.

11 Results and Interpretation

All results in this section were produced by running the repository’s own examples, plus two additional studies that I wrote while preparing this document in order to separate diagnostic bias from solver error and to identify the mechanism of the Stefan failure.

11.1 Bubble growth

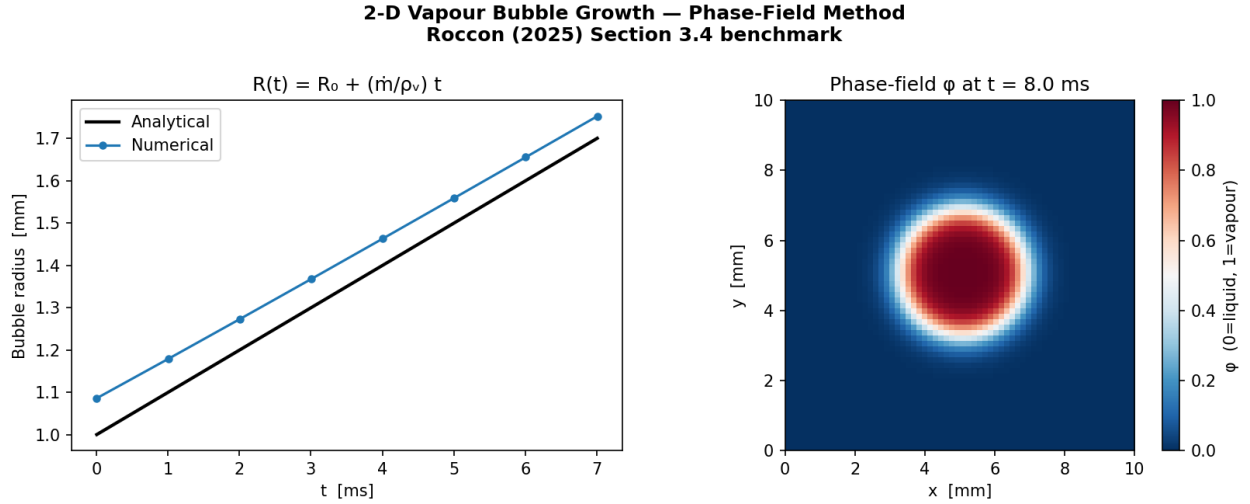


Figure 1: 2D vapour bubble growth. Left: bubble radius versus time, numerical (blue) against the analytical law $R(t) = R_0 + (\dot{m}/\rho_v)t$ (black). Right: the phase field ϕ at $t = 8$ ms, showing the bubble has retained its circular shape and a clean diffuse interface.

The run at 64×64 produces:

t [ms]	R_{num} [mm]	R_{ana} [mm]	error
0.01	1.0864	1.0005	8.58%
1.01	1.1793	1.1005	7.16%
2.01	1.2733	1.2005	6.06%
3.00	1.3680	1.3005	5.19%
4.00	1.4634	1.4005	4.49%
5.00	1.5594	1.5005	3.93%
6.00	1.6558	1.6005	3.46%
7.00	1.7526	1.7005	3.06%

The qualitative result is good: growth is linear, the bubble stays circular, the interface stays sharp, and the relative error *decreases* monotonically. But the numbers hide two distinct effects that a naive reading would conflate, and separating them is the useful part of this exercise.

The offset at $t = 0$ is a diagnostic artefact, not a solver error. The 8.58% error at the first output is present before the solver has done any meaningful work. Its origin is in `bubble_radius_2d`: the integral $\int \phi dA$ over a *diffuse* circular profile does not equal the area of the sharp circle of the same nominal radius. In polar geometry the tanh tail lying outside $r = R_0$ occupies more area

(its annuli are larger) than the deficit inside, so the integral over-estimates. Evaluating the initial condition alone, with no time stepping at all:

N	Δx [m]	ε [m]	R measured [mm]	bias
32	3.125×10^{-4}	4.687×10^{-4}	1.2931	+29.31%
64	1.563×10^{-4}	2.344×10^{-4}	1.0859	+8.59%
128	7.813×10^{-5}	1.172×10^{-4}	1.0223	+2.23%
256	3.906×10^{-5}	5.859×10^{-5}	1.0056	+0.56%

The bias falls by successive factors of 3.41, 3.85 and 3.98 as ε is halved — converging on 4, i.e. it is $\mathcal{O}(\varepsilon^2)$ — and it accounts for essentially the whole of the $t = 0$ discrepancy (8.59% predicted, 8.58% observed). It is a property of the *measurement* combined with the finite interface thickness — precisely the $\mathcal{O}(\varepsilon)$ modelling error that the asymptotic analysis of Section 4.4 predicts, made visible.

The growth rate carries the real solver error. Separating the slope from the offset by running a grid-refinement study on the growth rate itself:

N	Δt [s]	dR/dt [m/s]	slope error	final radius error
32	5×10^{-6}	0.08680	−13.20%	+10.42%
64	5×10^{-6}	0.09541	−4.59%	+2.73%
128	1×10^{-6}	0.09870	−1.30%	+0.66%

against the exact $\dot{m}/\rho_v = 0.1$ m/s. The time step is reduced at $N = 128$ to stay inside the viscous stability limit of Section 8.5, which falls as Δx^2 and equals 1.53×10^{-6} s at that resolution; re-running $N = 128$ at $\Delta t = 2 \times 10^{-6}$ s (marginally above the limit) changes the slope error only from −1.30% to −1.34%, so the trend below is a genuine spatial-convergence result and not an artefact of the varying Δt .

Both errors converge cleanly: the slope error by factors of 2.88 and 3.53, the final-radius error by 3.82 and 4.14 — approximately **second-order convergence** in Δx (with $\varepsilon \propto \Delta x$). This is exactly what the second-order central differencing and the $\mathcal{O}(\varepsilon^2)$ modelling error together predict, and it reproduces Roccon’s own observation for this benchmark that “by increasing the grid resolution, the agreement between numerical and analytical results improves and, for the largest grid resolution considered, the two lines perfectly overlap.”

The two errors have **opposite signs** — the measurement over-estimates R , the solver under-estimates dR/dt — which is why the reported error in the 64×64 table decreases with time: the initial positive bias is slowly eaten away by the accumulating negative slope error. That cancellation is fortuitous and should not be mistaken for convergence in time.

The residual growth-rate deficit at fixed resolution has three plausible contributors, listed in what I judge to be decreasing order of importance: the omitted phase-change momentum term (Defect 2, Section 9.5), which acts to retard the expansion flow; periodic confinement, which opposes expansion in a way that outflow boundary conditions would not; and the undersized γ (Defect 1, Section 9.1), which lets the interface smear and therefore mis-locates where the source $\dot{m}''' \propto \phi(1-\phi)$ is deposited. Distinguishing these is a well-defined next experiment: implementing the conservative momentum advection alone should shift the slope, and the magnitude of that shift would settle the question.

Verdict. The bubble benchmark is a genuine success. It validates the mass source (27), the divergence constraint (28), the constant-coefficient Poisson formulation (43), and the FFT solver, at a density ratio of 10^{-3} , with demonstrated second-order convergence.

11.2 The Stefan problem

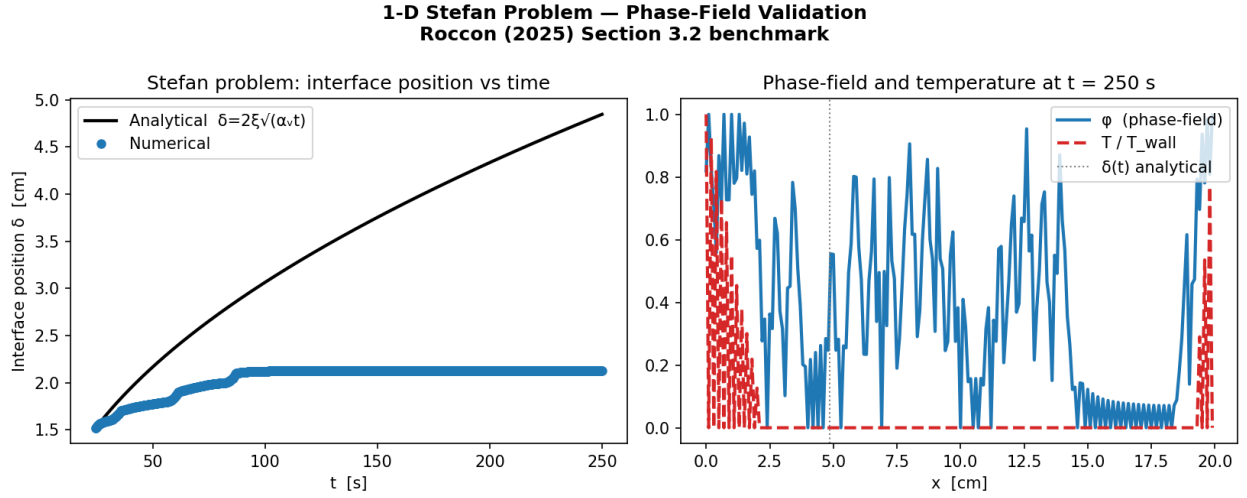


Figure 2: 1D Stefan problem. Left: interface position $\delta(t)$, numerical (blue) against the analytical similarity solution $\delta = 2\xi\sqrt{\alpha_v t}$ (black); the numerical front stalls near 2.1 cm. Right: the phase field (blue) and normalised temperature (red) at $t = 250$ s, showing complete loss of monotonicity across the domain.

This benchmark fails, and the figure shows it unambiguously. The interface tracks the similarity solution briefly and then stalls near 2.1 cm while the analytical solution continues to 4.84 cm — a 56% error at $t = 250$ s. The right-hand panel shows why any comparison beyond the early transient is meaningless: the phase field has disintegrated into grid-scale oscillations spanning the entire domain, and the temperature field shows matching oscillatory spikes at *both* ends.

I instrumented the run to establish the mechanism, tracking the interface error alongside two additional diagnostics: the number of $\phi = 0.5$ crossings in the domain (a count of spurious interfaces), and the integral $\int \phi dx$ (the total vapour content, which for a clean single front equals δ).

t [s]	δ_{num} [m]	δ_{ana} [m]	error	$\int \phi dx$ [m]	crossings
25	0.01530	0.01532	0.12%	0.01582	1
26	0.01556	0.01562	0.40%	0.01639	2
30	0.01596	0.01678	4.89%	0.01804	2
40	0.01729	0.01938	10.78%	0.02060	2
60	0.01854	0.02374	21.87%	0.02544	2
80	0.01995	0.02741	27.21%	0.02927	2
100	0.02123	0.03064	30.73%	0.03260	6
150	0.02125	0.03753	43.38%	0.05616	20
200	0.02125	0.04333	50.96%	0.07129	34
250	0.02130	0.04840	56.02%	—	—

This separates the failure into two distinct phases.

Phase 1 ($t \lesssim 100$ s): systematic under-prediction. The error grows smoothly from 0.12% to 31% with the field still essentially clean (2 crossings, the second being the unavoidable periodic wrap-around). The interface consistently *lags* the analytical solution. This is the signature of an under-estimated vaporisation rate, and its cause is the missing probe method. The energy step imposes `T_new = np.where(phi_new < 0.5, p.T_sat, T_new)`, so the temperature is clamped to T_{sat} everywhere on the liquid side. The vapour-side gradient at the interface is then evaluated with a *central* difference that straddles that clamp, sampling one node of genuine superheated vapour and one node of clamped liquid. The result is approximately half the true one-sided gradient — so $\dot{m}$ is under-estimated by roughly a factor of two, and the front lags. This is exactly the deficiency the repository README already identifies, and the trace confirms it quantitatively.

Phase 2 ($t \gtrsim 100$ s): numerical disintegration. From $t = 100$ s the reported δ_{num} *freezes* at 0.02125 m while $\int \phi dx$ keeps climbing — reaching 0.0713 m at $t = 200$ s, which is 65% *more* vapour than the analytical $\delta = 0.0433$ m. The crossing count explodes from 6 to 34. The interpretation is unambiguous: vaporisation is still occurring, indeed over-vigorously, but the vapour is being deposited as spurious blobs scattered through the domain rather than at the front, and `interface_position_1d`, which returns the *first* crossing, has locked onto a spurious one.

The mechanism is a closed feedback loop with four links, each of which is individually defensible and which together are fatal:

1. **The saturation clamp creates a C^0 discontinuity in T .** Overwriting $T = T_{\text{sat}}$ wherever $\phi < 0.5$ inserts a kink at the interface whose location snaps between grid cells as the front advances.
2. **Central differences have zero dissipation at the $2\Delta x$ mode.** As established in Section 8.1, the modified wavenumber $\sin(kh)/h$ vanishes at $k = \pi/h$. Differencing a discontinuity excites the Nyquist mode, and nothing in the scheme damps it.
3. **The interface normal amplifies the oscillation.** The vaporisation rate uses `nx = dphidx / (|dphidx| + 1e-14)`, a pure sign function. Once ϕ carries even a small $2\Delta x$ ripple, $\partial_x \phi$ changes sign at every local extremum, so $\hat{n}$ flips, and the source $\dot{m}''' \propto \hat{n}$ alternates between vaporisation and condensation on adjacent cells. This converts a small ripple into a large, sign-alternating source.
4. **The variable diffusivity closes the loop.** The example uses `alpha_f = alpha_v * phi`, so an oscillatory ϕ produces an oscillatory diffusion coefficient, and $\partial_x(\alpha \partial_x T)$ with a checkerboard α on a collocated grid is a textbook odd–even decoupling generator. That feeds fresh oscillation back into T , and hence into link 1.

Two further factors accelerate the collapse. The `np.clip(phi, 0, 1)` in the phase-field step masks the unboundedness — the trace confirms ϕ stays within $[0, 1]$ throughout — but converts it into a non-smooth, clipped field rather than preventing it, so the symptom that the boundedness theory is designed to warn about is suppressed. And the periodic boundary conditions are simply wrong for this problem: the wall at $x = 0$ is wrapped onto the outlet at $x = L_x$, which is why a second interface exists from $t = 26$ s and why the temperature field in the figure shows spikes at *both* ends. The wall Dirichlet condition `T_new[0,0] = p.T_wall` is imposed at a single node whose left neighbour, under `np.roll`, is the rightmost cell of the domain.

Verdict. The Stefan benchmark is not currently validated, and Section 12 records it as such. The value of the run is diagnostic: it isolates two required corrections — the probe method for one-sided gradients (Phase 1) and non-periodic boundary conditions plus an oscillation-control strategy

(Phase 2) — and gives a quantitative target, since Phase 1 error should fall below a few percent once probes are in place.

It is worth being explicit that this is a failure of *this prototype*, not of the method. Roccon (2025) reports “an excellent agreement among present results, the analytical profiles and the archival literature data of Sun et al. for the entire range of density ratios considered” for exactly this benchmark, using the full probe method, a staggered grid, and proper wall/outlet boundary conditions. The gap between his result and this one is a precise inventory of what this prototype still lacks.

12 Limitations

This section consolidates the known limitations, combining those documented in the repository README with the defects identified in Section 9 and the failure analysis of Section 11.2. They are ordered by the priority I assign them for the next stage of work.

12.1 Vaporisation rate: no probe interpolation, and a broken library function

The heat-flux closure evaluates temperature gradients at grid points with central differences rather than at probe points $\mathbf{x}_i \pm \Delta \mathbf{n}$ from the $\phi = 0.5$ iso-contour (Roccon 2025, Section 2.4). Section 11.2 quantifies the consequence: a systematic under-prediction of $\dot{m}$ reaching 31% error in interface position by $t = 100$ s in the Stefan problem, because a central difference straddling the saturation clamp returns roughly half the true one-sided gradient.

Compounding this, `src/energy.py::mdot_from_heatflux_2d` is incorrect as written (Defect 5, Section 9.6): it factors the two one-sided gradients into a single central one, returning $(k_v - k_l) \nabla T \cdot \hat{\mathbf{n}} / h_{lv}$. This is identically zero when $k_v = k_l$ — the Stefan configuration — and has the wrong sign for the default water properties. The function is currently never exercised, because the Stefan example inlines its own correct single-sided form. This is the highest-priority fix.

12.2 Explicit surface tension and the time-step restriction

The bubble benchmark runs with $\sigma = 0$. The justification recorded in the code and README is a time-step restriction of order 10^{-9} s, and it is worth correcting the arithmetic since the number matters for planning.

The Brackbill–Kothe–Zemach limit (9) at the bubble-benchmark resolution, with $\rho_l + \rho_v \approx 1001$ kg/m³, $\Delta x = 1.5625 \times 10^{-4}$ m, $\sigma = 0.07$ N/m, gives $\Delta t_\sigma \approx 9.3 \times 10^{-5}$ s — about fifteen times *larger* than the binding viscous limit of 6.1×10^{-6} s computed in Section 8.5. The 10^{-9} s figure appears to come from using ρ_v alone in place of the density sum; even that variant, evaluated literally, gives $\approx 7.4 \times 10^{-6}$ s — of the same order as the viscous limit, and some seven thousand times larger than 10^{-9} s.

So surface tension is **not** the binding stability constraint at this resolution, and the stated reason for setting $\sigma = 0$ is not the operative one. The actual justification is the sound one already given in the example’s own comment: the analytical solution (57) is independent of σ , so setting it to zero isolates the mass-source physics under test. The real difficulties with explicit CSF here are **spurious currents** — which do not disappear as Δt shrinks and which would contaminate a clean measurement of the growth rate — and the fact that (9) scales as $\Delta x^{3/2}$, so it *will* become binding at DNS resolutions. Implicit or semi-implicit surface tension remains a genuine requirement; the justification for it should just be stated accurately.

12.3 Periodic boundary conditions only

`np.roll` gives periodicity everywhere, and the FFT Poisson solver assumes it. This is elegant for a triply-periodic box and disqualifying for anything else. Section 11.2 shows it actively corrupting the Stefan problem, where the heated wall at $x = 0$ is wrapped onto the outlet at $x = L_x$, producing a second interface from $t = 26$ s and temperature spikes at both ends of the domain.

The eventual PhD target — nucleate boiling in a wall-bounded turbulent channel — is fundamentally incompatible with periodicity in the wall-normal direction. This requires both new boundary

handling in the operators and a different pressure solver, discussed in Section 13.3.

12.4 No oscillation control

Central differencing is non-dissipative by design, which is correct for DNS, but it leaves the scheme with no mechanism to remove $2\Delta x$ modes once excited. Section 11.2 identifies the excitation mechanism in the Stefan problem — the saturation clamp discontinuity, amplified by the sign-function interface normal and by odd–even decoupling in the variable-diffusivity term. The `np.clip` on ϕ hides the symptom without addressing the cause, and in doing so removes the warning signal that a boundedness violation would otherwise provide.

The properly-sized γ of Section 9.1 is part of the answer: a sharpening term at $\gamma = |\mathbf{u}|_{\max}$ is designed to be “the stiffest term in the equation” (Jain, 2022) and actively restores the tanh profile, which suppresses ripples. The collocated-versus-staggered grid choice (Section 8.1) is the other part.

12.5 First-order explicit time integration

All equations use explicit Euler, matching Roccon (2025), who also uses a first-order projection–correction method and explicit Euler for the phase-field and energy equations. This is adequate for the steady or self-similar benchmarks implemented here, where the solution is smooth in time and the error is dominated by spatial resolution.

It will not be adequate for turbulent DNS, where temporal accuracy directly affects the energy cascade, and where the accumulated phase error of a first-order scheme over $\mathcal{O}(10^5)$ steps is unacceptable. Note also from Section 8.5 that the bubble benchmark already runs at 82% of the explicit viscous stability limit, so there is little headroom.

12.6 Structural defects carried forward

For completeness, the defects identified in Section 9 that are not covered above:

- **Defect 1** — `gamma = eps` is a units error; γ is a velocity scale and is currently $\sim 200\times$ smaller than the boundedness criterion (24) requires.
- **Defect 2** — momentum advection uses the non-conservative form, omitting the phase-change momentum term $\mathbf{u} \dot{m}''' (1 - \rho_v/\rho_l)$.
- **Defect 3** — the 3D viscous term drops the variable-viscosity correction the 2D version retains; both drop the $\nabla \mathbf{u}^T$ contribution. Dormant only because both benchmarks use matched viscosities.
- **Defect 4** — the implemented S_t does not match Eq. (R.11); it should either be implemented faithfully or omitted explicitly per Roccon’s saturation argument.
- **Defect 6** — the velocity clamp uses `p.Lx / p.dt` where the comment says one grid cell, making it inert.
- **Defect 7** — the 3D energy path is unreachable dead code.
- The energy advection term is non-conservative where Eq. (R.9) specifies conservative form.
- The README’s 3D snippet uses `indexing='ij'`, inconsistent with the `(Nz, Ny, Nx)` layout the 3D operators assume.
- `examples/stefan_1d.py` attributes its $t_0 = 24.7$ s start time to Roccon’s Section 3.2; it belongs to Section 3.3.

12.7 Collocated rather than staggered grid

Roccon (2025) Section 2.5 uses a staggered arrangement with velocities at cell faces and scalars at cell centres, as do FLOW36 and MHIT36. This repository is collocated. Collocated grids permit odd–even pressure–velocity decoupling, and the Stefan failure of Section 11.2 involves exactly that mechanism in the variable-diffusivity term. This is a structural divergence from the reference method that should be closed before the port, since the production framework is staggered anyway.

12.8 Python performance

NumPy is entirely adequate for 2D prototyping: the 64×64 bubble benchmark completes 1600 steps in about 30 seconds, and the 200-point Stefan problem runs 450,000 steps in roughly five minutes. It is not a production tool. A turbulent boiling DNS at 512^3 or beyond requires Fortran with GPU offload, which is the entire point of the port described in Section 13.5. This prototype is a validation instrument, and the studies in Section 11 are the kind of work it is for.

13 Future Directions and PhD Roadmap

13.1 Immediate corrections

Ordered by priority, these are the fixes that should precede any new physics:

1. **Implement the probe method** (Roccon 2025 Section 2.4). Locate the $\phi = 0.5$ iso-contour using the signed-distance property, place probes at $\mathbf{x}_i \pm \Delta \mathbf{n}$ with $\Delta \sim \Delta x$, interpolate T to each probe, form the one-sided gradients, and evaluate (37). This addresses the dominant Stefan error directly and also repairs Defect 5.
2. **Fix γ** to a velocity scale satisfying (24), ideally recomputed as $|\mathbf{u}|_{\max}$ each step. Then remove the `np.clip` and verify that boundedness holds *analytically*, as Mirjalili et al. (2020) guarantee — which converts the clip from a crutch into an assertion.
3. **Conservative momentum advection**, removing Defect 2, and measure the resulting change in the bubble growth rate. This is a clean, decisive experiment for attributing the residual slope error identified in Section 11.1.
4. **Unify the 2D and 3D viscous terms** and implement the full $\nabla \cdot [\mu(\nabla \mathbf{u} + \nabla \mathbf{u}^T)]$, removing Defect 3, before any run with realistic viscosity contrast.
5. Resolve Defects 4, 6, 7 and the documentation inconsistencies listed in Section 12.6.

13.2 Three-dimensional validation

All `*_3d` operators, `ac_rhs_3d`, `ns_step_3d`, `solve_poisson_3d` and `run_3d` are written; none has been validated. The natural first test is the 3D analogue of the bubble benchmark, for which the analytical solution is the same law (57) — the derivation of Section 10.1 in spherical geometry gives $dV/dt = \dot{m} 4\pi R^2 / \rho_v$ with $V = \frac{4}{3}\pi R^3$, hence again $dR/dt = \dot{m} / \rho_v$. The diagnostic `bubble_radius_3d` is already in place, and the $\mathcal{O}(\varepsilon^2)$ measurement bias characterised in Section 11.1 should be re-measured in spherical geometry before any error is attributed to the solver.

A 64^3 run at the bubble-benchmark parameters is well within NumPy’s reach and would validate the 3D operator set, the 3D FFT Poisson solve, and the 3D coupling — the entire Year-1 3D task — on a laptop. The `indexing='ij'` inconsistency noted in Section 9.2 must be fixed first.

13.3 Wall and outlet boundary conditions

This is the structural change with the widest consequences, and it is required before any wall-bounded turbulence work.

Periodicity must be replaced with, at minimum: no-slip and a temperature Dirichlet or Neumann condition at the heated wall; and an outflow condition ($p = 0$, $\partial \mathbf{u} / \partial n = 0$) at the opposite boundary, as Roccon uses in his Sections 3.1–3.4. This requires ghost-cell or one-sided stencils in `operators.py` in place of `np.roll`.

It also requires a different pressure solver, because the FFT eigen-decomposition of Section 8.3 relies on periodicity. Three viable options:

- **Sine/cosine transforms.** A homogeneous Dirichlet condition diagonalises under the discrete sine transform, a homogeneous Neumann condition under the discrete cosine transform. Both are available as `scipy.fft.dst/dct` and as FFT-based kernels on GPU, so the $\mathcal{O}(N \log N)$ direct-solve property is retained exactly. This is the natural choice for one non-periodic direction.

- **FFT in the periodic directions plus a tridiagonal solve in the non-periodic one.** Transform in x and z , leaving an independent tridiagonal system in y for each wavenumber pair, solved by the Thomas algorithm. This is the standard approach for channel flow and is what FLOW36 effectively does, with Chebyshev polynomials in the wall-normal direction.
- **Iterative (multigrid/Krylov).** Fully general, but sacrifices exactly the property that motivated the method, and scales worst on GPUs. To be avoided if either of the above applies.

The second option is the one to aim for, since it matches the eventual FLOW36 architecture.

13.4 Higher-order time integration

Replace explicit Euler with a low-storage third-order Runge–Kutta scheme (Williamson or RK3-Wray), the standard choice in DNS, retaining explicit treatment of advection and possibly moving the viscous and surface-tension terms to an implicit or semi-implicit treatment. The two motivations are accuracy in the turbulent cascade and relief of the $\Delta t \sim \Delta x^2$ viscous restriction, which Section 8.5 shows is already the binding constraint. Implicit surface tension (Section 12.2) belongs in the same work package.

13.5 The Fortran/CUDA/MPI port

The production target is a boiling-capable fork of FLOW36 (Roccon, Soligo and Soldati, 2025), with reference to MHIT36 (Roccon et al., 2025) for the finite-difference GPU patterns. The additions required over the existing non-boiling codes are, precisely: the energy equation (34) with mixture diffusivity (35); the vaporisation source in the phase-field equation (26); the mass source in the pressure Poisson right-hand side (43); and the probe-based Rankine–Hugoniot closure (37). Every one of these is small in isolation. The engineering effort is in the parallel decomposition of the probe search — which is the only genuinely non-local operation in the whole algorithm, and the only one that does not map trivially onto a halo exchange.

Working through the probe algorithm’s data dependencies in the Python prototype first, and in particular establishing the maximum stencil radius the interpolation requires, is therefore directly load-bearing for the port. That is a concrete argument for doing item 1 of Section 13.1 carefully rather than quickly.

13.6 Physics and parameter studies

Once the solver is validated in 3D with wall boundary conditions:

- **Film boiling**, named by Roccon (2025) as the immediate next benchmark, and a natural bridge between the validated single-bubble cases and full nucleate boiling.
- **Nucleate boiling in a turbulent channel** — the central PhD simulation. This requires a nucleation-site model, since the phase-field method as formulated captures the growth of an existing vapour region but does not spontaneously nucleate one.
- **Nusselt-number scaling.** `diagnostics.py::nusselt_number_2d` is a stub awaiting the wall boundary conditions of Section 13.3. The target is a scaling law $Nu = f(Re, Ja, \rho_v/\rho_l)$ across the parameter space, with the Jakob number $Ja = C_{p,l}\Delta T/h_{lv}$ measuring superheat — the quantity that empirical boiling correlations attempt to capture and that a DNS can address from first principles.
- **Density-ratio sweep.** Roccon validates from $\rho_v/\rho_l = 1$ down to 5×10^{-4} ; the same range should be re-established in 3D and in turbulence, since boundedness of ϕ is the property most

likely to fail first as the ratio drops.

13.7 Roadmap summary

Priority	Task	Depends on	Section
1	Probe method for $\dot{m}$; fix Defect 5	—	13.1
2	Fix γ ; remove <code>np.clip</code> crutch	—	13.1
3	Conservative momentum advection	—	13.1
4	3D bubble benchmark at 64^3	1–3	13.2
5	Wall/outlet BCs; DST/DCT or tridiagonal solver	4	13.3
6	RK3 time integration; implicit surface tension	5	13.4
7	Fortran + CUDA/MPI port onto FLOW36	4–6	13.5
8	Film boiling; nucleate boiling in turbulence	7	13.6
9	Nusselt scaling; density-ratio sweep	8	13.6

14 References

- Allen, S. M., and Cahn, J. W. (1979). A microscopic theory for antiphase boundary motion and its application to antiphase domain coarsening. *Acta Metallurgica* **27**(6), 1085–1095. DOI: 10.1016/0001-6160(79)90196-2
- Brackbill, J. U., Kothe, D. B., and Zemach, C. (1992). A continuum method for modeling surface tension. *Journal of Computational Physics* **100**(2), 335–354. DOI: 10.1016/0021-9991(92)90240-Y
- Cahn, J. W., and Hilliard, J. E. (1958). Free energy of a nonuniform system. I. Interfacial free energy. *The Journal of Chemical Physics* **28**(2), 258–267. DOI: 10.1063/1.1744102
- Chiu, P.-H., and Lin, Y.-T. (2011). A conservative phase field method for solving incompressible two-phase flows. *Journal of Computational Physics* **230**(1), 185–204. DOI: 10.1016/j.jcp.2010.09.021
- Chorin, A. J. (1968). Numerical solution of the Navier–Stokes equations. *Mathematics of Computation* **22**(104), 745–762. DOI: 10.1090/S0025-5718-1968-0242392-2
- Esmaeeli, A., and Tryggvason, G. (2004). Computations of film boiling. Part I: numerical method. *International Journal of Heat and Mass Transfer* **47**(25), 5451–5461. DOI: 10.1016/j.ijheatmasstransfer.2004.07.027
- Gibou, F., Chen, L., Nguyen, D., and Banerjee, S. (2007). A level set based sharp interface method for the multiphase incompressible Navier–Stokes equations with phase change. *Journal of Computational Physics* **222**(2), 536–555. DOI: 10.1016/j.jcp.2006.07.035
- Giustini, G., and Issa, R. I. (2021). A method for simulating interfacial mass transfer on arbitrary meshes. *Physics of Fluids* **33**(8), 087102. DOI: 10.1063/5.0058987
- Haghani-Hassan-Abadi, R., Fakhari, A., and Rahimian, M.-H. (2021). Phase-change modeling based on a novel conservative phase-field method. *Journal of Computational Physics* **432**, 110111. DOI: 10.1016/j.jcp.2021.110111
- Hirt, C. W., and Nichols, B. D. (1981). Volume of fluid (VOF) method for the dynamics of free boundaries. *Journal of Computational Physics* **39**(1), 201–225. DOI: 10.1016/0021-9991(81)90145-5
- Irfan, M., and Muradoglu, M. (2017). A front tracking method for direct numerical simulation of evaporation process in a multiphase system. *Journal of Computational Physics* **337**, 132–153. DOI: 10.1016/j.jcp.2017.02.036
- Jacqmin, D. (1999). Calculation of two-phase Navier–Stokes flows using phase-field modeling. *Journal of Computational Physics* **155**(1), 96–127. DOI: 10.1006/jcph.1999.6332
- Jain, S. S. (2022). Accurate conservative phase-field method for simulation of two-phase flows. *Journal of Computational Physics* **469**, 111529. DOI: 10.1016/j.jcp.2022.111529
- Jain, S. S., Mani, A., and Moin, P. (2020). A conservative diffuse-interface method for compressible two-phase flows. *Journal of Computational Physics* **418**, 109606. DOI: 10.1016/j.jcp.2020.109606
- Juric, D., and Tryggvason, G. (1998). Computations of boiling flows. *International Journal of Multiphase Flow* **24**(3), 387–410. DOI: 10.1016/S0301-9322(97)00050-5

- Kharangate, C. R., and Mudawar, I. (2017). Review of computational studies on boiling and condensation. *International Journal of Heat and Mass Transfer* **108**, 1164–1196. DOI: 10.1016/j.ijheatmasstransfer.2016.12.065
- Kunkelmann, C., and Stephan, P. (2009). CFD simulation of boiling flows using the volume-of-fluid method within OpenFOAM. *Numerical Heat Transfer, Part A* **56**(8), 631–646. DOI: 10.1080/10407780903423908
- Mangani, F., Soligo, G., Roccon, A., and Soldati, A. (2022). Influence of density and viscosity on deformation, breakage, and coalescence of bubbles in turbulence. *Physical Review Fluids* **7**, 053601. DOI: 10.1103/PhysRevFluids.7.053601
- Mirjalili, S., Ivey, C. B., and Mani, A. (2020). A conservative diffuse interface method for two-phase flows with provable boundedness properties. *Journal of Computational Physics* **401**, 109006. DOI: 10.1016/j.jcp.2019.109006. Preprint: arXiv:1803.01262
- Mirjalili, S., Khanwale, M. A., and Mani, A. (2023). Assessment of an energy-based surface tension model for simulation of two-phase flows using second-order phase field methods. *Journal of Computational Physics* **474**, 111795. DOI: 10.1016/j.jcp.2022.111795
- Mohammadi-Shad, M., and Lee, T. (2017). Phase-field lattice Boltzmann modeling of boiling using a sharp-interface energy solver. *Physical Review E* **96**(1), 013306. DOI: 10.1103/PhysRevE.96.013306
- Olsson, E., and Kreiss, G. (2005). A conservative level set method for two phase flow. *Journal of Computational Physics* **210**(1), 225–246. DOI: 10.1016/j.jcp.2005.04.007
- Osher, S., and Sethian, J. A. (1988). Fronts propagating with curvature-dependent speed: algorithms based on Hamilton–Jacobi formulations. *Journal of Computational Physics* **79**(1), 12–49. DOI: 10.1016/0021-9991(88)90002-2
- Roccon, A. (2025). Boiling heat transfer by phase-field method. *Acta Mechanica* 236, 5623–5638. DOI: 10.1007/s00707-024-04122-7** — the primary reference for this work; open access under CC-BY 4.0.
- Roccon, A., Enzenberger, L., Zaza, D., and Soldati, A. (2025). MHIT36: A phase-field code for GPU simulations of multiphase homogeneous isotropic turbulence. *Computer Physics Communications* **314**, 109804. DOI: 10.1016/j.cpc.2025.109804
- Roccon, A., Soligo, G., and Soldati, A. (2025). FLOW36: A spectral solver for phase-field based multiphase turbulence simulations on heterogeneous computing architectures. *Computer Physics Communications* **313**, 109640. DOI: 10.1016/j.cpc.2025.109640
- Roccon, A., Zonta, F., and Soldati, A. (2023). Phase-field modeling of complex interface dynamics in drop-laden turbulence. *Physical Review Fluids* **8**, 090501. DOI: 10.1103/PhysRevFluids.8.090501
- Son, G., and Dhir, V. K. (1998). Numerical simulation of film boiling near critical pressures with a level set method. *Journal of Heat Transfer* **120**(1), 183–192. DOI: 10.1115/1.2830042
- Sun, D., Xu, J., and Chen, Q. (2014). Modeling of the evaporation and condensation phase-change problems with FLUENT. *Numerical Heat Transfer, Part B: Fundamentals* **66**(4), 326–342. DOI: 10.1080/10407790.2014.915681
- Sun, Y., and Beckermann, C. (2004). Diffuse interface modeling of two-phase flows based on averaging: mass and momentum equations. *Physica D* **198**(3–4), 281–308. DOI:

10.1016/j.physd.2004.09.003

Tamura, A., and Katono, K. (2022). Development of a phase-field method for phase change simulations using a conservative Allen–Cahn equation. *Journal of Nuclear Engineering and Radiation Science* **8**(3), 031402. DOI: 10.1115/1.4052807

Tanguy, S., Ménard, T., and Berlemont, A. (2007). A level set method for vaporizing two-phase flows. *Journal of Computational Physics* **221**(2), 837–853. DOI: 10.1016/j.jcp.2006.07.003

Tanguy, S., Sagan, M., Lalanne, B., Couderc, F., and Colin, C. (2014). Benchmarks and numerical methods for the simulation of boiling flows. *Journal of Computational Physics* **264**, 1–22. DOI: 10.1016/j.jcp.2014.01.014

Tryggvason, G., Scardovelli, R., and Zaleski, S. (2011). *Direct Numerical Simulations of Gas–Liquid Multiphase Flows*. Cambridge University Press, Cambridge. DOI: 10.1017/CBO9780511975264

Udaykumar, H. S., Shyy, W., and Rao, M. M. (1996). ELAFINT: a mixed Eulerian–Lagrangian method for fluid flows with complex and moving boundaries. *International Journal for Numerical Methods in Fluids* **22**(8), 691–712. DOI: 10.1002/(SICI)1097-0363(19960430)22:8<691::AID-FLD371>3.0.CO;2-U

Unverdi, S. O., and Tryggvason, G. (1992). A front-tracking method for viscous, incompressible, multi-fluid flows. *Journal of Computational Physics* **100**(1), 25–37. DOI: 10.1016/0021-9991(92)90307-K

Wang, Z., Zheng, X., Chrysosostomidis, C., and Karniadakis, G. E. (2021). A phase-field method for boiling heat transfer. *Journal of Computational Physics* **435**, 110239. DOI: 10.1016/j.jcp.2021.110239

Welch, S. W. J., and Wilson, J. (2000). A volume of fluid based method for fluid flows with phase change. *Journal of Computational Physics* **160**(2), 662–682. DOI: 10.1006/jcph.2000.6481

14.1 Software

FLOW36 — pseudo-spectral solver for DNS of multiphase turbulence, MultiphaseFlowLab, University of Udine / TU Wien. <https://github.com/MultiphaseFlowLab/FLOW36>

MHIT36 — multi-GPU phase-field code for multiphase homogeneous isotropic turbulence. <https://github.com/MultiphaseFlowLab/MHIT36>

boiling-phasefield-3d — the Python prototype documented here.

14.2 Data availability

The data supporting Roccon (2025) are openly available in the Figshare repository linked from that article. All numerical results reported in Section 11 of this document were produced by running `examples/bubble_2d.py` and `examples/stefan_1d.py` from this repository, together with two additional instrumented scripts (the grid-refinement study of Section 11.1 and the Stefan diagnostic trace of Section 11.2) whose parameters are stated in full in those sections.